UNIVERZITET U NIŠU

ELEKTRONSKI FAKULTET U NIŠU

Master akademske studije

Računarstvo i informatika

Modul: Softversko inženjerstvo

Seminarski rad iz predmeta

Sistemi za upravljanje bazama podataka

Interna struktura i organizacija indeksa u Firebird bazi podataka

Profesor:

dr Aleksandar S. Stanimirović

Student:

Jefimija Stamenović 1820/24

Mart, 2026. godine

Sadržaj

1. Uvod 4
   1.1. Kontekst indeksa u modernim sistemima za upravljanje bazama podataka 4
   1.2. Istorijski razvoj Firebird baze podataka i evolucija On-Disk Structure (ODS) 4
   1.3. Definicija problema: Balansiranje između brzine pretrage i opterećenja pri upisu 5
   1.4. Metodologija rada i struktura istraživanja 6
2. Fizička arhitektura i organizacija skladištenja 7
   2.1. On-Disk Structure (ODS): Koncept stranica fiksne veličine 7
   2.2. Tipovi stranica baze podataka 8
   2.3. Unutrašnja struktura stranice sa podacima – Data Page 8
      2.3.1. Zaglavlje stranice i mehanizmi validacije integriteta 8
      2.3.2. Tabela slotova i indirektno adresiranje zapisa 9
      2.3.3. Upravljanje slobodnim prostorom i fragmentacija unutar bloka 9
   2.4. Sistemski identifikator RDB$DB_KEY 9
3. Multi-generational architecture (MGA) i dinamika verzija 11
   3.1. Dinamika modifikacije zapisa i alokacija verzija 11
   3.2. Čuvanje indeksnih unosa i kooperativno čišćenje (Garbage Collector) 12
   3.3. Vidljivost transakcija i Transaction Inventory Page (TIP) 12
   3.4. Fenomen Index Bloat i fizička degradacija struktura 12
4. Teorijske osnove i struktura indeksa 14
   4.1. Algoritam B+ stabla: Logička organizacija i balansiranje 14
   4.2. Specijalizovane indeksne stranice u Firebird-u 14
      4.2.1. Index Root Page: Registar i metapodaci indeksa 14
      4.2.2. Index B-tree page: Hijerarhija navigacionih čvorova 15
   4.3. Struktura stranica na nivou lista: Mapiranje ključa na RDB$DB_KEY 15
   4.4. Dinamičko upravljanje prostorom: Operacija Page Split i Fill Factor 16
5. Teorijske osnove i struktura indeksa 17
   5.1. Mehanizam prefiksne kompresije unutar indeksne stranice 17
   5.2. Upravljanje duplikatima i NULL vrednostima 18
      5.2.1. Grupisanje duplikata i kompresija adresa 18
      5.2.2. Indeksiranje NULL vrednosti 19

5.3. Binarna reprezentacija različitih tipova podataka u ključu 20
6. Tipovi indeksa i njihova primena 20
6.1. Primarni, unikatni i strani ključevi (integritetni indeksi) 20
6.2. Indeksi nad izrazima (Expression Indices) i njihova interna perzistencija 21
6.3. Smer sortiranja i kompozitni indeksi 22
7. Statistika indeksa 23
7.1. Selektivnost indeksa i matematički model procene troška 23
7.2. Uticaj komande SET STATISTICS na preciznost plana upita 24
7.3. Strategije optimizatora: Index Scan, Index Seek i Full Table Scan 24
8. Dinamička dijagnostika i monitoring indeksnih operacija u realnom vremenu 26
8.1. Interpretacija PLAN naredbe i razumevanje putanje upita 26
8.2. Index Navigation: Optimizacija za ORDER BY i GROUP BY klauzule 28
9. Praktična evaluacija i dijagnostika indeksnih operacija u realnom vremenu 29
10. Zaključak 40
11. Literatura 41

1. Uvod

1.1. Kontekst indeksa u modernim sistemima za upravljanje bazama podataka

U savremenom inženjerstvu baza podataka, brzina pronalaženja informacija u relacionim bazama podataka predstavlja osnovni parametar skalabilnosti sistema. Kako digitalna transformacija poslovanja dovodi do eksponencijalnog rasta količine uskladištenih podataka, tabele koje sadrže desetine miliona zapisa postaju standard čak i u srednjim preduzećima. U takvim okolnostima, oslanjanje na sekvencijalno skeniranje celokupne tabele (engl. *Full Table Scan*) rezultira neprihvatljivim kašnjenjima, jer vreme izvršavanja upita raste linearno sa brojem zapisa, što u praksi dovodi do zagušenja I/O podsistema i procesorskih resursa

Indeksi su uvedeni kao specijalizovane, pomoćne strukture podataka čija je primarna uloga radikalna transformacija složenosti pretrage. Umesto pretraživanja svake pojedinačne stranice na kojoj se nalaze podaci, sistem koristi indeks kao sortiranu mapu ključeva koja direktno referencira fizičke lokacije unutar datoteke baze podataka. Sa matematičke tačke gledišta, pravilno implementiran indeks svodi linearnu složenost $O(n)$ na logaritamsku $O(\log n)$, što omogućava da se željeni podatak locira u svega nekoliko diskretnih koraka, bez obzira na ukupnu veličinu tabele.

Međutim, indeksiranje u modernim RDBMS sistemima nije statičan proces već kompleksna inženjerska trgovina (engl. *trade-off*). Svaki dodatni indeks zahteva trajni skladišni prostor na disku i aktivno trošenje memorijskih resursa unutar keša baze podataka. Balansiranje između brzine odziva na upite i cene održavanja integriteta podataka predstavlja centralni zadatak dizajnera baza podataka, jer prekomerno indeksiranje može dovesti do značajnog usporavanja operacija modifikacije podataka, stvarajući usko grlo u sistemima sa visokom frekvencijom transakcija.

1.2. Istorijski razvoj Firebird baze podataka i evolucija On-Disk Structure (ODS)

Koreni Firebird-a proizilaze iz 2000. godine, kada je kompanija Borland donela stratešku odluku o objavljivanju izvornog koda svog InterBase 6.0 sistema pod otvorenom licencom tj. postaje *open-source*. Ovaj čin nije bio samo čin ustupanja koda, već je postavio temelj za razvoj zajednice koja je nasledila jednu od najnaprednijih arhitektura tog vremena – sistem koji je već tada nudio potpunu podršku za viševerzijsku kontrolu konkurentnosti (MVCC). Firebird je tokom naredne dve decenije evoluirao iz InterBase derivata u potpuno nezavisan i robustan sistem za upravljanje bazama podataka koji se danas odlikuje specifičnim balansom između niske potrošnje resursa i sposobnosti da upravlja ekstremno kompleksnim relacionim šemama. Njegova arhitektura je od samog početka bila podređena autonomiji – cilj je bio kreirati bazu koja zahteva minimalne intervencije od strane administratora (DBA), a pritom zadržava integritet podataka u uslovima naglog prekida napajanja ili hardverskog otkaza.

Fizički integritet i logika smeštanja podataka u Firebirdu definisani su kroz koncept *On-Disk Structure* (ODS). ODS nije ništa drugo do binarni protokol koji diktira kako će se bajtovi organizovati na medijumu, gde će stajati metapodaci, a gde stvarni podaci i njihovi pripadajući indeksi. Svaki put kada se o otvori `.fdb` datoteka, *Engine* prvo proverava verziju ODS-a kako bi utvrdio koji skup internih pravila i algoritama treba primeniti. Ova struktura je ključna jer

obezbeđuje da baza ostane prenosiva – datoteka baze podataka se može jednostavno kopirati sa Windows na Linux platform pod uslovom da verzija Engine-a prepoznaje ODS broj. Kroz razvoj Firebirda, ODS je bio poligon za uvođenje inovacija u načinu na koji se podaci čitaju i pišu, što je pratilo napredak samih hardverskih medija, od mehaničkih diskova do savremenih NVMe drajvova.

Tokom decenija, svaka značajna promena verzije ODS-a (od ODS 11 do aktuelnih standarda u verzijama 4 i 5) bila je primarno motivisana potrebom za probijanjem fizičkih ograničenja prethodnih generacija. Rane verzije su se suočavale sa limitima u pogledu maksimalne dužine ključa i broja indeksnih segmenata, što je ograničavalo upotrebu baze u sistemima sa kompleksnim primarnim ključevima. Moderni ODS standardi su ove barijere srušili uvođenjem 64-bitnog adresiranja, čime je omogućeno da baza efikasno mapira ogromne količine podataka bez gubitka performansi pri navigaciji. Poseban fokus stavljen je na prefiksnu kompresiju unutar indeksnih stranica, čime je gustina podataka po stranici drastično povećana, omogućavajući Firebirdu da ostane izuzetno brz čak i kada veličina baze pređe terabajtne okvire.

Međutim, razvoj ODS-a nije se zaustavio samo na povećanju skladišnog kapaciteta, već se prava revolucija desila u optimizaciji putanje pretrage. Svaka nova iteracija sistema donela je poboljšanja u algoritmima za balansiranje stabala, sa jasnim ciljem – smanjenje "dubine" indeksa. U inženjerskom smislu, dubina predstavlja broj stranica koje Engine mora da preuzme sa diska u radnu memoriju da bi locirao specifičan zapis. Smanjenjem dubine se direktno štedi na I/O operacijama, što je najskuplji resurs u radu sa bazama. Danas, zahvaljujući evoluciji ODS-a, Firebird može da obradi kompleksne upite i pretrage nad funkcijama (*Expression Indexes*) sa efikasnošću koja je ranije bila rezervisana samo za skupa komercijalna rešenja, pružajući programerima slobodu da modeluju podatke bez straha od usporavanja indeksnog podsistema.

1.3. Definicija problema: Balansiranje između brzine pretrage i opterećenja pri upisu

Srž inženjerskog problema koji ovaj rad istražuje leži u direktnom sukobu između brzine pronalaženja informacija i cene koju sistem plaća prilikom svake modifikacije baze. U bilo kom relacionom sistemu, indeks nije statičan entitet već živa struktura koja zahteva neprekidno održavanje u realnom vremenu. Svaka operacija koja menja stanje podataka — bilo da je reč o prostom unosu novog reda ili ažuriranju postojeće vrednosti — pokreće lančanu reakciju unutar svih definisanih indeksnih stabala te tabele. U scenarijima gde tabela poseduje veći broj indeksa (npr. preko pet ili deset), trošak održavanja referencijalnog integriteta i sortnog redosleda može postati dominantan faktor u trajanju transakcije. Umesto jedne I/O operacije na nivou stranice podataka, Engine biva primoran da izvrši na desetine dodatnih upisa u različite indeksne stranice, što u sistemima sa visokim stepenom konkurentnosti dovodi do ozbiljne degradacije performansi i povećanja latencije.

U Firebird-u je ovaj problem dodatno intenziviran njegovom unutrašnjom logikom rada, specifično kroz ***Multi-Generational Architecture*** (MGA). Za razliku od baza koje modifikuju podatke "u mestu" (engl. *in-place update*), Firebird kreira potpuno nove verzije zapisa za svaku promenu. To praktično znači da svaki `UPDATE` operacija za motor baze predstavlja kombinaciju brisanja i ponovnog unosa, što duplira opterećenje nad indeksnim strukturama. Indeksni podsistem ne samo da mora da umetne pokazivač na novu verziju, već mora da zadrži i validnost pokazivača na stare verzije koje su možda još uvek potrebne transakcijama koje su u toku. Ovakva dinamika

rada direktno utiče na fizičku stabilnost indeksa, jer svaka promena ključnog polja zahteva rebalansiranje čitavih grana B+ stabla, trošeći dragocene resurse procesora i memorijskog keša.

Kao direktna posledica ovakve arhitekture javlja se fenomen "naduvanosti" indeksa (engl. *Index Bloat*), koji predstavlja jedan od najvećih izazova u administraciji Firebird baza. Pošto sistem sledi MVCC model, indeksne stranice se popunjavaju pokazivačima na verzije zapisa koji su hronološki zastareli, ali fizički još uvek prisutni na disku. Rezultat je paradoksalna situacija gde indeks zauzima višestruko više prostora nego što bi to zahtevala stvarna količina unikatnih ključeva. Ovakav rast direktno kvari gustinu podataka po stranici i povećava dubinu stabla, čime se direktno poništava primarna svrha indeksa — brzina. Projektovanje indeksne strategije stoga postaje kompleksna potraga za "zlatnom sredinom" tj kako pokriti sve kritične upite bez uvođenja nepotrebnih struktura koje bi sistem učinile inertnim i neefikasnim u uslovima masovnog unosa podataka.

Ovaj rad se stoga fokusira na dekonstrukciju procesa kojima se ovi negativni efekti mogu kontrolisati ili eliminisati. Istraživanje obuhvata analizu uticaja parametara kao što je *Page Size*, koji direktno određuje koliko "široko" stablo može biti, kao i ulogu selektivnosti koja diktira da li će optimizator uopšte uzeti indeks u obzir. Problem se ne posmatra samo kroz prizmu brzine izvršavanja pojedinačnog upita, već kroz prizmu ukupnog protoka sistema (engl. *throughput*). Cilj je definisati metodologiju koja minimizuje broj fizičkih I/O ciklusa po svakoj transakciji, uz održavanje visoke preciznosti navigacije kroz podatke, što je preduslov za generisanje stabilnih i optimalnih planova izvršavanja u produkcionim okruženjima

1.4. Metodologija rada i struktura istraživanja

Ovaj rad je zasnovan na analizi unutrašnjih procesa Firebird-a, sa fokusom na uzročno-posledične veze između fizičkog zapisa na disku i brzine izvršavanja upita. Umesto teorijskih opisa, fokus je na tome kako Engine zapravo manipuliše bajtovima i zašto određena podešavanja, koja na papiru deluju beznačajno, u praksi mogu da prepolove ili dupliraju vreme odziva baze. Istraživanje je podeljeno tako da prati putanju podatka od njegovog fizičkog smeštanja do momenta kada ga optimizator pronađe putem indeksa.

Prvi deo rada se bavi fizičkom arhitekturom baze — stranicama i slotovima. Bez razumevanja kako Firebird deli prostor u `.fdb` datoteci i kako dodeljuje adrese (RDB$DB_KEY), nemoguće je shvatiti zašto se indeksi ponašaju na određeni način. Zatim prelazimo na analizu B+ stabala i mehanizama kompresije, gde istražujemo kako sistem pokušava da uštedi prostor i smanji broj čitanja sa diska. Na kraju, kroz praktične testove, merimo kako sve te unutrašnje strukture reaguju na realna opterećenja, posebno u situacijama kada ima previše indeksa ili kada je statistika baze zastarela.

2. Fizička arhitektura i organizacija skladištenja

Da bi se precizno definisao mehanizam funkcionisanja indeksa, neophodno je prvo analizirati fizički sloj baze podataka. Firebird bazu podataka (.fdb) ne treba posmatrati kao skup logičkih entiteta, već kao niz binarnih blokova fiksne dužine koji su smešteni unutar datoteke na disku. Svaka operacija pretraživanja, u svojoj suštini, predstavlja proces navigacije kroz ove blokove, gde arhitektura stranica direktno diktira brzinu odziva sistema

2.1. On-Disk Structure (ODS): Koncept stranica fiksne veličine

Svaka Firebird baza podataka je zapravo jedna datoteka podeljena na jednake delove koji se nazivaju stranice. Prilikom inicijalizacije svake baze, administrator definiše parametar *Page Size* (najčešće 8 KB ili 16 KB), koji postaje fiksni okvir za sve operacije. Firebird ne vrši čitanje podataka na nivou pojedinačnih bajtova, već uvek dovlači kompletnu stranicu u memorijski keš (*Buffer Pool*). Veličina stranice ima kritičan uticaj na performanse indeksa. Budući da se indeksna stabla skladište unutar istih tih stranica, veći *Page Size* omogućava smeštanje većeg broja ključeva u jedan čvor. Rezultat je manja "dubina" indeksa što znači da Engine mora izvršiti manje fizičkih I/O operacija po disku da bi stigao do željenog zapisa. U bazama sa velikim brojem redova, smanjenje dubine indeksa za samo jedan nivo može eliminisati milione nepotrebnih operacija čitanja.

Sistem *On-Disk Structure* (ODS) predstavlja skup pravila i standarda koji govore *engine*-u gde se u tim stranicama šta nalazi. Svaka stranica ima svoje fiksno mesto i svoj identifikacioni broj. Za proces indeksiranja je ključno to što su stranice fiksne veličine, jer to omogućava bazi da momentalno izračuna gde se u fajlu nalazi bilo koja stranica. Ako sistemu treba stranica broj 500, on ne gubi vreme tražeći je – on vrši direktno adresiranje na osnovu matematičkog modela:

$$fizički_adresa = broj_stranice\ x\ veličina_stranice$$

Sledeći SQL upit demonstrira kako se iz baze očitavaju definisana veličina stranice (Page Size) i verzija ODS-a:

Izvorni kod: Upit za ekstrakciju parametara fizičkog sloja baze

```
1  SELECT
2      MON$DATABASE_NAME AS DB_PATH,
3      MON$ODS_MAJOR || '.' || MON$ODS_MINOR AS ODS_VERSION,
4      MON$PAGE_SIZE AS PAGE_SIZE,
5      MON$PAGES AS TOTAL_PAGES
6  FROM MON$DATABASE;
```

2.2. Tipovi stranica baze podataka

Da bi sistem znao kako da tretira bajtove učitane u memoriju, prva lokacija na svakoj stranici nosi identifikacioni kod tipa. Ovi kodovi omogućavaju Engine-u da trenutno razlikuje metapodatke od stvarnog sadržaja. Iako postoji čitav niz specijalizovanih stranica, za proces indeksiranja su od ključne važnosti sledeća četiri tipa:

- **Tip 0x01 (Header Page) –** prva stranica u datoteci koja čuva vitalne informacije: ODS verziju, veličinu stranice i ID poslednje transakcije
- **Tip 0x05 (Data Page) –** fizički kontejner u kojem se čuvaju stvarni podaci (redovi tabela).
- **Tip 0x06 (Index Root Page)** – katalog svih indeksa definisanih za određenu tabelu. Ova stranica ne sadrži ključeve, već pokazivače na lokacije gde počinju pripadajuća indeksna stabla.
- **Tip 0x07 (Index B-Tree Page)** – sama unutrašnjost indeksa, gde su smešteni sortirani ključevi i navigacioni pokazivači ka podacima

Ova podela osigurava da se navigacione strukture drže odvojeno od samog "tereta" podataka, što omogućava bazi da pretraži milione ključeva bez učitavanja ijedne stranice sa podacima sve do samog finala operacije.

2.3. Unutrašnja struktura stranice sa podacima – Data Page

Stranica sa podacima predstavlja osnovnu operativnu jedinicu unutar koje Firebird vrši skladištenje podataka. Njena unutrašnja organizacija nije linearna, već je dizajnirana kao visoko optimizovan dinamički prostor koji mora da ispuni dva suprotstavljena zahteva:

- efikasno korišćenje dostupnih bajtova
- očuvanje fiksnih adresa za potrebe indeksiranja

Svaka stranica funkcioniše kao nezavisan kontejner koji poseduje sopstvenu logiku upravljanja slobodnim prostorom, što omogućava Engine-u da vrši lokalne modifikacije bez uticaja na globalnu strukturu datoteke.

2.3.1. Zaglavlje stranice i mehanizmi validacije integriteta

Na samom početku svakog bloka nalazi se fiksni skup metapodataka koji definiše identitet i stanje stranice. Pored osnovnih informacija o tipu stranice i njenoj pripadnosti određenoj tabeli, najkritičniji element zaglavlja je kontrolni zbir (engl. *Checksum*). Ovaj mehanizam funkcioniše tako što sistem, prilikom svakog upisivanja stranice iz memorijskog keša na disk, izračunava specifičan matematički potpis svih bajtova na toj stranici i skladišti ga u zaglavlje.

Kada se stranica naknadno učitava, Firebird momentalno ponavlja ovaj proračun i upoređuje rezultat sa vrednošću iz zaglavlja. Ukoliko postoji i najmanja razlika među njima, sistem to detektuje kao fizičko oštećenje (engl. *Page Corruption*). Ovakva rigidna provera je nezaobilazna jer indeksi, kao kompleksne strukture pokazivača, zavise od apsolutne tačnosti fizičkih adresa. Bez validacije integriteta na nivou svake pojedinačne stranice, postojala bi opasnost od tihe korupcije podataka, gde bi neispravan bajt u indeksnom čvoru mogao da dovede do potpunog gubitka pristupa čitavim granama podataka ili do pogrešnih rezultata upita.

2.3.2. Tabela slotova i indirektno adresiranje zapisa

Najvažniji gradivni element koji omogućava funkcionisanje indeksa jeste tabela slotova (engl. *Line Index*), smeštena na samom kraju stranice. Dok se stvarni podaci (redovi tabela) smeštaju odmah nakon zaglavlja i ređaju ka dnu, tabela slotova se popunjava u obrnutom smeru – od poslednjeg bajta stranice ka njenoj sredini. Svaki unos u ovoj tabeli predstavlja "slot" koji sadrži dve ključne informacije: precizan pomeraj (engl. *offset*) od početka stranice do mesta gde zapis zaista počinje i ukupnu dužinu tog zapisa u bajtovima.

Ovakav sistem indirektnog mapiranja je ključan za stabilnost baze. Kada se podatak u tabeli ažurira i njegova dužina promeni, Engine ga može pomeriti na bilo koju slobodnu lokaciju unutar te iste stranice radi optimizacije prostora. Međutim, broj slota u tabeli na dnu ostaje nepromenjen. Za indeksne strukture koje pokazuju na ovaj zapis, on se i dalje nalazi na istoj "adresi", iako se fizički pomerio za nekoliko stotina bajtova unutar samog bloka. Da nema ovog sistema slotova, svaka i najmanja promena dužine podatka zahtevala bi hitno ažuriranje svih indeksa koji referenciraju taj red, što bi u sistemima sa visokim intenzitetom izmena dovelo do potpunog kolapsa performansi usled masovnog zaključavanja i I/O zagušenja.

2.3.3. Upravljanje slobodnim prostorom i fragmentacija unutar bloka

Unutar svake stranice sa podacima postoji prostor koji još uvek nije zauzet zapisima ili tabelom slotova, poznat kao "slobodni prostor" (engl. *Free Space*). Firebird konstantno prati veličinu ovog prostora kako bi znao da li na tu konkretnu stranicu može da smesti novu verziju zapisa ili novi red. On-Disk struktura zahteva da se podaci unutar stranice drže u što kompaktnijem stanju. Kada se neki zapis obriše, on ostavlja "rupu" na stranici. Motor baze te rupe ne popunjava odmah, već čeka pogodan trenutak ili proces reorganizacije da bi izvršio sabijanje (engl. *Garbage Collection* na nivou stranice), čime se slobodan prostor ponovo okuplja u jednu celinu.

Ovaj mehanizam je direktno povezan sa efikasnošću indeksa jer određuje koliko će se često morati alocirati nove stranice u bazi. Ako je unutrašnja fragmentacija stranica visoka, baza raste brže nego što je potrebno, što povećava broj stranica koje indeksi moraju da prate. Efikasno upravljanje slotovima i prostorom unutar svake stranice tipa 0x05 osigurava da gustina podataka ostane visoka, što indirektno utiče na keširanje: više korisnih podataka u jednoj stranici znači manji broj čitanja sa diska i bolju iskorišćenost radne memorije servera.

2.4. Sistemski identifikator RDB$DB_KEY

Sada dolazimo do onoga što povezuje logičke upite i fizičku lokaciju podataka na disku, a to je **RDB$DB_KEY**. Dok programeri i administratori baza podataka operišu sa primarnim ključevima koje sami definišu, *engine* interno ne prepoznaje te vrednosti kao direktne adrese. Umesto toga, svaka pretraga preko indeksa se u finalnoj fazi svodi na generisanje ovog sistemskog identifikatora. On predstavlja osmobajtnu vrednost koja nosi apsolutnu informaciju o lokaciji zapisa, funkcionišući kao jedinstvena poštanska adresa unutar .fdb datoteke. Bez ovog mehanizma, proces lociranja podataka bi zahtevao konstantno i skupo pretraživanje sadržaja stranica, što bi u potpunosti poništilo prednosti indeksiranja.

Struktura RDB$DB_KEY identifikatora je strogo podeljena na dve komponente od po četiri bajta, koje direktno odgovaraju hijerarhiji stranica koju smo prethodno analizirali:

- **Prva četiri bajta – Redni broj stranice** (engl. *Page Number*) – pomoću ove vrednosti, Firebird vrši trenutno pozicioniranje na fizički pomeraj unutar datoteke, koristeći ranije objašnjenu formulu množenja broja stranice sa njenom fiksnom veličinom.
- **Druga četiri bajta - Broj slota** (engl. *Line Number* ili *Slot Index*) – redni broju u tabeli slotova na stranici. Ova vrednost upućuje Engine na tačan ulaz u tabeli slotova na dnu stranice, odakle se očitava finalni offset unutar samog bloka.

Ovakva dvo-komponentna struktura omogućava da se podatak locira u svega nekoliko procesorskih ciklusa, bez obzira na to da li se baza prostire na nekoliko megabajta ili više terabajta.

Značaj RDB$DB_KEY identifikatora je najočigledniji u kontekstu performansi indeksiranja i stabilnosti sistema tokom masovnih operacija. Budući da indeksi u svojim listovima (engl. *Leaf nodes*) čuvaju upravo ove identifikatore, a ne same vrednosti podataka ili logičke ključeve, postignuta je maksimalna brzina u završnom koraku izvršavanja upita. Čim pretraga kroz B+ stablo indeksa dođe do tražene vrednosti, ona isporučuje RDB$DB_KEY optimizatoru, koji zatim vrši direktan pristup (engl. *Direct Access*) memorijskoj stranici. Važno je istaći da je ovaj ključ specifičan za životni ciklus jedne baze i da se ne može koristiti za poređenje podataka između dve različite datoteke. Njegova uloga je isključivo unutrašnja navigacija koja garantuje da put od indeksnog čvora do fizičkog bita na disku bude najkraći mogući, čime se Firebird pozicionira kao sistem sa izuzetno niskim nadzemnim troškovima (engl. *overhead*) pri radu sa velikim setovima podataka.

U specifičnim situacijama, **RDB$DB_KEY** se može koristiti i direktno u SQL upitima kako bi se postiglo najbrže moguće čitanje, zaobilazeći čak i standardnu logiku indeksa. Iako se ovo u praksi retko koristi zbog zavisnosti od fizičke strukture, to ilustruje koliko je ovaj identifikator "blizu" samom hardveru. U kontekstu seminara, razumevanje ovog ključa je presudno jer on objašnjava zašto je važno održavati integritet stranica i slotova – svaka promena u RDB$DB_KEY adresi značila bi da svi indeksi u bazi postaju nevalidni. Upravo ta fiksna veza između broja stranice i slota na dnu stranice tipa 0x05 omogućava Firebirdu da zadrži ekstremnu brzinu pretrage, istovremeno podržavajući kompleksne operacije modifikacije podataka kroz svoju MVCC arhitekturu.

Direktan uvid u ove interne fizičke adrese moguće je ostvariti kroz SQL upit nad bilo kojom tabelom, koristeći implicitnu kolonu RDB$DB_KEY. Sledeći primer prikazuje binarne identifikatore zapisa, koji za Engine predstavljaju finalnu destinaciju nakon navigacije kroz B+ stablo indeksa:

Izvorni kod: Prikaz internih fizičkih adresa zapisa (RDB$DB_KEY)

```
7   /* Ekstrakcija sistemskih identifikatora koji povezuju
8      logički zapis sa fizičkom lokacijom na disk stranici */
9   SELECT
10      RDB$DB_KEY,
11      EMP_NO,
12      LAST_NAME
13  FROM EMPLOYEE
14  WHERE DEPT_NO = '622'; -- Primer filtriranja
15
```

3. Multi-generational architecture (MGA) i dinamika verzija

Arhitektura Firebird sistema je zasnovana na principu **Multi-Generational Architecture (MGA)**, koji predstavlja specifičnu primenu **MVCC (Multi-Version Concurrency Control)** kontrole. Za razliku od sistema koji integritet podataka čuvaju mehanizmima zaključavanja (engl. locking) tabela ili redova, Firebird omogućava istovremeni rad više korisnika tako što operacije čitanja nikada ne blokiraju operacije upisa. Ovaj model direktno utiče na organizaciju indeksa, jer svaka izmena u bazi dovodi do čuvanja više različitih stanja istog zapisa unutar strukture stranica.

3.1. Dinamika modifikacije zapisa i alokacija verzija

Unutar Firebird motora, operacija ažuriranja podataka (UPDATE) ne podrazumeva izmenu postojećih binarnih podataka na njihovoj originalnoj fizičkoj lokaciji. Umesto izmene na samom mestu gde se podatak nalazi, sistem vrši upis nove instance zapisa na prvoj slobodnoj lokaciji unutar stranica podataka (tip 0x05). Originalni zapis se zadržava kao prethodna verzija (engl. *Back-version*), čime se formira hronološki lanac verzija koji je povezan pokazivačima u zaglavljima zapisa.

Ovakva dinamika iz korena menja ulogu indeksa u procesu pronalaženja podataka. Indeksna struktura ne mapira logički ključ na jednu statičnu tačku, već isključivo na **početak lanca verzija (engl. *version chain*)**. Svaki RDB$DB_KEY koji indeks dostavi zapravo predstavlja polaznu koordinatu za kretanje kroz hronologiju zapisa. Ukoliko transakcija koja vrši upit nema pravo pristupa najnovijoj izmeni — što zavisi od njenog nivoa izolacije — Engine je prinuđen da prati pomenuti lanac verzija unazad kroz prethodne zapise. Shodno tome, efikasnost indeksa nije određena samo brzinom pretrage unutar B+ stabla, već i ukupnom dužinom lanca verzija koji se mora obraditi nakon što je fizička adresa pronađena.

Sledeći primer praktično ilustruje lanac verzija. Kroz seriju uzastopnih izmena nad istim zaposlenim, mi zapravo ne menjamo fizički postojeći slog, već kreiramo lanac različitih verzija tog reda koje se skladište jedna iza druge. Iako smo tri puta promenili podatak, RDB$DB_KEY (fizička adresa slota na stranici baze) ostaje nepromenjena, što potvrđuje da indeks i dalje pokazuje na istu lokaciju, ali se unutar te lokacije nalazi "istorija" verzija.

Izvorni kod: Vizualizacija lanca verzija

```
16 /* Generisanje lanca verzija nad istim redom */
17 UPDATE EMPLOYEE SET PHONE_EXT = 'V1' WHERE EMP_NO = 2;
18 COMMIT;
19 UPDATE EMPLOYEE SET PHONE_EXT = 'V2' WHERE EMP_NO = 2;
20 COMMIT;
21 UPDATE EMPLOYEE SET PHONE_EXT = 'V3' WHERE EMP_NO = 2;
22 COMMIT;
23
24 /* Provera trenutnog stanja - vidimo samo najnoviju verziju */
25 SELECT EMP_NO, PHONE_EXT, RDB$DB_KEY
26 FROM EMPLOYEE
27 WHERE EMP_NO = 2;
```

3.2. Čuvanje indeksnih unosa i kooperativno čišćenje (Garbage Collector)

Firebird koristi specifičan model čišćenja zastarelih podataka koji se naziva **kooperativni Garbage Collection (GC)**. Ovaj proces se ne pokreće kao poseban program u pozadini, već ga iniciraju sami korisnički upiti. Kada SQL upit prolazi kroz lanac verzija tražeći aktuelni podatak, on usput prepoznaje one verzije koje više nisu potrebne nijednoj aktivnoj transakciji i označava ih za brisanje. Tada se njihovo mesto na stranicama sa podacima (tip 0x05) oslobađa za nove upise.

Glavni problem se javlja unutar samih indeksa (stranice tipa 0x07). Dok se podaci sa običnih stranica obrišu relativno brzo, unosi u indeksima često ostaju upisani sve dok se cela stranica indeksa ne prepakuje ili dok se indeks ne rekreira. To stvara situaciju u kojoj indeks i dalje sadrži putokaze koji vode ka mestima gde podataka više nema. Engine je tada primoran da učitava stranice sa diska samo da bi ustanovio da je taj podatak obrisan. To nepotrebno troši vreme procesora i usporava rad baze, naročito u tabelama sa visokom frekvencijom izmena.

3.3. Vidljivost transakcija i Transaction Inventory Page (TIP)

Da bi baza znala koji je podatak iz indeksa ispravan za određenu transakciju, ona mora da proveri status transakcija na posebnim **TIP stranicama**. Svaki zapis u bazi u svom zaglavlju nosi broj transakcije koja ga je napravila. Kada indeks pronađe fizičku adresu, Engine prvo očitava taj broj, a zatim proverava na TIP stranici da li je ta transakcija potvrđena (*Committed*) ili poništena (*Rolled Back*).

Ovaj proces postaje usko grlo u sistemima gde postoji neka veoma stara transakcija koja je ostala otvorena (engl. *Oldest Active Transaction*). Prema pravilima MGA arhitekture, baza ne sme da obriše nijednu verziju podatka koja je nastala nakon te najstarije aktivne transakcije. Rezultat je nagomilavanje neobrisanih verzija unutar indeksa, što povećava broj poređenja koje baza mora da uradi. Indeksni sistem je, dakle, direktno uslovljen efikasnošću upravljanja transakcijama na nivou aplikacije.

3.4. Fenomen Index Bloat i fizička degradacija struktura

Kao posledica stalnog kreiranja verzija i nepotpunog čišćenja, javlja se pojava poznata kao **Index Bloat** (naduvanost indeksa). Usled čestih modifikacija, indeksne stranice dostižu svoj maksimalni kapacitet što pokreće proces deljenja stranica (engl. *Page Split*). Iako se broj stvarnih redova u tabeli možda ne menja, broj indeksnih stranica i dubina stabla stalno rastu.

Fizička fragmentacija dovodi do toga da indeks zauzima prostor koji je višestruko veći od optimalnog. Pošto Firebird ne vrši automatsko sažimanje B+ stabla u realnom vremenu, jedini efikasan metod za rešavanje ovog problema je **ponovna izgradnja indeksa**. Procesom prebacivanja indeksa u neaktivno stanje **(ALTER INDEX INACTIVE)**, a potom ponovnim aktiviranjem, stablo se gradi iznova. Na taj način se odbacuju sve suvišne verzije i postiže se optimalna popunjenost stranica, što vraća indeks u stanje maksimalne efikasnosti i smanjuje broj potrebnih operacija na disku.

Pogledajmo naredni primer. U prvom delu, upitom nad tabelom **RDB$INDICES** očitavamo statistiku indeksa gde niska vrednost u koloni **RDB$STATISTICS** ukazuje na visoku efikasnost.

Uzastopnim izvršavanjem deset UPDATE operacija svesno kreiramo veliki broj verzija zapisa u bazi, što primorava *engine* da održava dugačke lance verzija i privremeno narušava performanse.

Izvorni kod: Index bloat

```
28 /* Korak 1: Provera selektivnosti indeksa PRE bloat-a */
29 SELECT RDB$INDEX_NAME, RDB$STATISTICS AS SELEKTIVNOST
30 FROM RDB$INDICES
31 WHERE RDB$RELATION_NAME = 'EMPLOYEE'
32 ORDER BY RDB$INDEX_NAME;
33
34 /* Korak 2: Generisanje bloat-a - 10 uzastopnih izmjena */
35 UPDATE EMPLOYEE SET SALARY = SALARY * 1.01 WHERE DEPT_NO = '621';
36 COMMIT;
37 UPDATE EMPLOYEE SET SALARY = SALARY * 1.01 WHERE DEPT_NO = '621';
38 COMMIT;
39 UPDATE EMPLOYEE SET SALARY = SALARY * 1.01 WHERE DEPT_NO = '621';
40 COMMIT;
41 UPDATE EMPLOYEE SET SALARY = SALARY * 1.01 WHERE DEPT_NO = '621';
42 COMMIT;
43 UPDATE EMPLOYEE SET SALARY = SALARY * 1.01 WHERE DEPT_NO = '621';
44 COMMIT;
45 UPDATE EMPLOYEE SET SALARY = SALARY * 1.01 WHERE DEPT_NO = '621';
46 COMMIT;
47 UPDATE EMPLOYEE SET SALARY = SALARY * 1.01 WHERE DEPT_NO = '621';
48 COMMIT;
49 UPDATE EMPLOYEE SET SALARY = SALARY * 1.01 WHERE DEPT_NO = '621';
50 COMMIT;
51 UPDATE EMPLOYEE SET SALARY = SALARY * 1.01 WHERE DEPT_NO = '621';
52 COMMIT;
53 UPDATE EMPLOYEE SET SALARY = SALARY * 1.01 WHERE DEPT_NO = '621';
54 COMMIT;
```

Komandom gfix -sweep pokreće proces fizičkog čišćenja baze (Garbage Collection), koji uklanja sve nepotrebne verzije zapisa koje više nijedna aktivna transakcija ne može videti. Ovim postupkom se baza vraća u optimalno stanje, oslobađa se prostor na stranicama podataka i osigurava da indeksi ponovo rade maksimalnom brzinom, što je od kritičnog značaja za održavanje performansi sinhronizovane replike.

Izvorni kod: Sweeping

```
55 gfix -sweep -user SYSDBA -password masterkey localhost:employee
```

Ponovnim izvršavanjem prvog SELECT-a ćemo videti da se statistika indeksa osvežila i da su opet efikasni.

Sve informacije o indeksu, možemo videti celu statistiku indeksa:

Izvorni kod: Informacije o indeksu

```
56 gstat -user SYSDBA -password masterkey localhost:employee
```

4. Teorijske osnove i struktura indeksa

Efikasnost Firebird sistema u obradi velikih skupova podataka zasniva se na strogoj hijerarhiji indeksnih struktura. Dok su stranice sa podacima zadužene za fizičko skladištenje zapisa, indeksne stranice formiraju logičku arhitekturu koja omogućava direktno adresiranje. Ova struktura transformiše proces pretraživanja iz linearnog ispitivanja svakog zapisa u preciznu navigaciju do konkretnih fizičkih lokacija unutar datoteke na disku.

4.1. Algoritam B+ stabla: Logička organizacija i balansiranje

Firebird indeksi su implementirani kao **B+ stabla**, što predstavlja varijaciju balansiranog stabla optimizovanu za rad sa stranicama fiksne veličine. Ključna karakteristika ovog algoritma je stroga podela uloga:

- svi ključevi i njihove pripadajuće fizičke adrese (RDB$DB_KEY) smešteni su isključivo u najnižem nivou — **listovima** (engl. *Leaf nodes*)
- viši nivoi služe isključivo za usmeravanje pretrage ka odgovarajućim granama

Balansiranje stabla osigurava da je putanja od korena do bilo kog lista uvek iste dubine. To znači da je broj I/O operacija potreban za pronalaženje bilo kog podatka konstantan. B+ stablo u Firebird-u raste vertikalno – kada se stranica na nivou lista popuni, ona se deli na dve, a informacija o toj podeli se prosleđuje korenu. Ukoliko se korenski čvor podeli, visina stabla se povećava za jedan nivo, čime se održava savršena simetrija i predvidljivost performansi.

4.2. Specijalizovane indeksne stranice u Firebird-u

Unutar ODS (On-Disk Structure) standarda, proces indeksiranja se oslanja na saradnju dva specifična tipa stranica koje čuvaju navigacione metapodatke i statističke parametre.

4.2.1. Index Root Page: Registar i metapodaci indeksa

Svaka tabela u Firebird bazi poseduje bar jednu **Index Root** stranicu. Ova stranica funkcioniše kao centralna evidencija svih indeksa definisanih nad tom tabelom. Ona ne sadrži same ključeve, već čuva sistemske parametre neophodne za inicijalizaciju pretrage:

- **Index ID –** jedinstveni identifikator indeksa unutar tabele.
- **Root Page Number –** fizički broj stranice na kojoj počinje korenski čvor B+ stabla.
- **Selectivity -** statistička vrednost (između 0 i 1) koja pokazuje koliko je indeks "jedinstven". Optimizator koristi ovu vrednost da odluči da li je korišćenje indeksa isplativije od skeniranja cele tabele.
- **Index Flags –** informacije o tome da li je indeks jedinstven (*unique*), uzlazni (*ascending*) ili silazni (*descending*)

U nastavku se nalazi SQL upit kojim se mogu videti metapodaci o indeksima iz sistemskog kataloga:

Izvorni kod: Upit za ekstrakciju metapodataka o indeksima iz sistemskog kataloga

```
SELECT
    RDB$INDEX_NAME AS INDEX_NAME,
    RDB$RELATION_NAME AS RELATION_TABLE,
    RDB$STATISTICS AS SELECTIVITY,
    RDB$INDEX_ID AS INDEX_ID
FROM RDB$INDICES
WHERE RDB$SYSTEM_FLAG = 0 -- Filtriramo samo korisničke indekse
ORDER BY RDB$RELATION_NAME;
```

4.2.2. Index B-tree page: Hijerarhija navigacionih čvorova

Indeksne stranice predstavljaju osnovne jedinice od kojih se gradi celokupno indeksno stablo. One su organizovane u strogu hijerarhiju i funkcionalno se dele na: **unutrašnje čvorove** i **listove**. Unutrašnji čvorovi (engl. *Non-leaf nodes*) služe isključivo za **strukturno upravljanje pretragom** i ne sadrže direktne reference ka podacima u tabelama. Njihov sadržaj čine parovi podataka koji se sastoje od vrednosti ključa i broja stranice koja se nalazi na sledećem, nižem nivou stabla. Unutar navigacionih čvorova, binarni podaci su raspoređeni u linearnom i strogo sortiranom nizu. Ovakva organizacija je ključna jer omogućava *engine-u* primenu **algoritma binarne pretrage** unutar memorijskog bloka same stranice. Umesto sekvencijalnog ispitivanja svakog unosa, sistem vrši logičku podelu opsega ključeva čime odmah može da identifikuje granu koja sadrži traženi opseg vrednosti. Glavna uloga unutrašnjih čvorova je brza redukcija prostora pretraživanja. Oni eliminišu čitave segmente stabla koji ne zadovoljavaju kriterijum upita što broj fizičkih čitanja sa diska svodi na logaritamsku zavisnost u odnosu na ukupan broj zapisa.

4.3. Struktura stranica na nivou lista: Mapiranje ključa na RDB$DB_KEY

Najniži hijerarhijski nivo B+ stabla čine **stranice na nivou lista** (engl. *Leaf pages*). Iako koriste istu strukturu bloka kao i unutrašnji čvorovi, one se suštinski razlikuju po svom sadržaju i specifičnom indikatoru u zaglavlju koji signalizira da je dostignuta završna tačka pretrage. Ovo je sloj arhitekture u kojem se vrši finalno povezivanje logičke vrednosti koju korisnik potražuje (poput primarnog ključa ili datuma) sa konkretnom fizičkom adresom zapisa unutar datoteke baze podataka.

Svaki unos na ovoj stranici je binarno optimizovan i sastoji se od dve ključne komponente:

1. **Vrednost ključa:** Binarna reprezentacija indeksiranog polja nad kojom je primenjen mehanizam prefiksne kompresije radi postizanja maksimalne gustine podataka.
2. **RDB$DB_KEY:** Osmobajtna koordinata koja predstavlja direktnu adresu. Prva četiri bajta definišu redni broj stranice sa podacima (tip 0x05), dok preostala četiri bajta služe kao indeks u tabeli slotova te stranice, što omogućava trenutni pristup traženom redu bez dodatnog pretraživanja unutar same Data stranice.

Specifičnost Firebird implementacije ogleda se u **horizontalnoj povezanosti** stranica na nivou lista. Svaka stranica u ovom sloju sadrži pokazivače na svog levog i desnog suseda, čime se formira

dvostruko povezana lista unutar samog stabla. Ova arhitektonska odluka je presudna za performanse upita koji zahtevaju sortiranje ili pretragu po opsegu (engl. *Range Scan*). Kada sistem jednom vertikalno navigira kroz stablo i locira početni ključ, on prekida vertikalno kretanje. Dalje procesiranje se vrši linearnim kretanjem kroz susedne stranice na nivou lista, čime se postiže maksimalna brzina čitanja sortiranih setova podataka uz minimalno angažovanje resursa za ponovnu navigaciju od korena stabla.

4.4. Dinamičko upravljanje prostorom: Operacija Page Split i Fill Factor

Budući da su stranice fiksne veličine, konstantno dodavanje novih ključeva (naročito usled MGA modela i generisanja verzija) zahteva mehanizam za dinamičko proširenje strukture. Kada motor baze pokuša da upiše novi ključ u stranicu koja nema dovoljno slobodnog prostora, aktivira se operacija **podeli stranicu** (engl. *Page Split*).

Ovaj proces se odvija kroz precizno definisane korake kako bi se očuvala ravnoteža stabla:

1. **Alokacija novog bloka:** Sistem pronalazi ili alocira novu praznu stranicu (0x07) unutar datoteke baze podataka.
2. **Redistribucija ključeva:** Približno polovina ključeva sa popunjene stranice seli se na novu stranicu. Ključno je da se ovaj prenos vrši tako da oba bloka ostanu sortirana i da se na oba mesta stvori prostor za buduće upise.
3. **Ažuriranje roditeljskog čvora:** U unutrašnjem čvoru na višem nivou dodaje se novi pokazivač koji referencira novonastalu stranicu. Ukoliko je i roditeljska stranica puna, proces se rekurzivno ponavlja naviše, što u krajnjem slučaju dovodi do podele korena i povećanja ukupne visine stabla.

Fill Factor (faktor popunjenosti) je parametar koji određuje koliki procenat stranice će biti zauzet podacima prilikom inicijalne izgradnje ili reindeksiranja. Firebird teži popunjenosti od 80% do 90%. Preostali prostor se ostavlja kao rezervni bafer za nove ključeve. Pravilno upravljanje ovim prostorom je ključno za sprečavanje preuranjene fragmentacije. Česta deljenja stranica direktno vode ka povećanju dubine stabla, što povećava broj I/O operacija i vreme odziva upita, stvarajući efekat koji smo ranije opisali kao *Index Bloat*.

5. Teorijske osnove i struktura indeksa

Efikasnost indeksa u Firebird sistemu ne zavisi isključivo od algoritma B+ stabla, već i od fizičke gustine zapisa unutar same stranice fiksne veličine. Povećanje broja ključeva po stranici direktno utiče na smanjenje visine B+ stabla, čime se minimizuje broj I/O operacija neophodnih za lociranje konkretnog podatka. Da bi se postigao optimalan faktor grananja (*fan-out*), *engine* primenjuje napredne mehanizme binarne optimizacije i diferencijalne kompresije na nivou bajtova.

5.1. Mehanizam prefiksne kompresije unutar indeksne stranice

Prefiksna kompresija je primarna tehnika za redukciju redundanse unutar sortiranih indeksnih struktura. Budući da su ključevi na svakoj stranici poređani po vrednosti, susedni zapisi u nizu često dele identične početne bajtove. Umesto skladištenja kompletnog ključa za svaki unos, Firebird beleži samo diferencijalnu promenu u odnosu na prethodni zapis na toj stranici.

Svaki indeksni unos definisan je preko dva kontrolna binarna parametra:

- **Prefix Count** – broj početnih bajtova koji su identični prethodnom ključu i koji se ne zapisuju ponovo.
- **Suffix Length** – dužina preostalog dela ključa koji predstavlja unikatnu promenu i koji se fizički smešta u blok podataka.

Primer binarne optimizacije: Razmotrimo indeks nad tekstualnim poljem sa sledećim sortiranim nizom:

- **Petrov** – (Prvi ključ u bloku: Prefix=0, Suffix=6, Podatak="Petrov")
- **Petrović** – (Prefix=6, Suffix=2, Podatak="ić")
- **Petronijević** – (Prefix=5, Suffix=7, Podatak="nijević")

Ovaj mehanizam omogućava da se, naročito kod složenih (kompozitnih) indeksa gde se vodeći segmenti (npr. datum ili šifra države) ponavljaju u hiljadama uzastopnih redova, ukupna fizička veličina indeksa smanji za više od 50%. Ovakva ušteda prostora održava stablo "plitkim", što je ključno za performanse na velikim setovima podataka.

Iako je ova ušteda prostora sjajna, ona uvodi jedan problem, a to je da se ne može prosto "uskočiti" u sredinu stranice i pročitati reč jer ona zavisi od svih reči iznad nje. Da bi se izbegao ovaj "domino efekat" i primorao bazu da čita sve redom, Firebird koristi **sidrišne tačke (Jump points)**.

Na svakih nekoliko zapisa, baza namerno upiše pun, nekompresovan ključ. Ova sidra služe kao markeri koji omogućavaju algoritmu da vrši brze binarne skokove unutar same stranice. Time je postignut balans između maksimalne gustine pakovanja i brzine pretrage u memoriji.

Primer ispod prikazuje kako Firebird efikasno upravlja indeksima nad tekstualnim kolonama. Prilikom kreiranja indeksa IDX_EMP_LASTNAME, sistem interno koristi prefiksnu kompresiju kojom eliminiše suvišno ponavljanje istih početnih karaktera unutar indeksnih stranica. Kada se izvrši upit sa klauzulom STARTING WITH, aktivira se **Range Scan** strategija. Zahvaljujući B+ stablo strukturi i kompresiji, baza ne pretražuje celu tabelu, već "skače" direktno na prvi list indeksa koji počinje slovom 'S' i linearno čita samo povezane susedne listove. Ovakav pristup drastično smanjuje broj potrebnih ulazno-izlaznih (I/O) operacija, što je ključno za održavanje visokih performansi u sistemima gde se vrše brze pretrage nad velikim bazama podataka.

Izvorni kod: Prefiksna kompresija

```
/* Upiti koji koriste prefiksnu kompresiju */
/* STARTING WITH aktivira Range Scan - baza odmah skače na odgovarajući list B+
stabla */
SET PLAN ON

SELECT EMP_NO, LAST_NAME, FIRST_NAME
FROM EMPLOYEE
WHERE LAST_NAME STARTING WITH 'S'
ORDER BY LAST_NAME;
/* Plan: INDEX (IDX_EMP_LASTNAME) Range Scan
   Prefiksna kompresija omogućava da svi prezimeni
   koji počinju sa 'S' budu na susjednim listovima */
SET PLAN OFF
```

5.2. Upravljanje duplikatima i NULL vrednostima

Efikasnost indeksa u Firebird sistemu ne ogleda se samo u brzini pretrage jedinstvenih vrednosti, već i u načinu na koji *engine* optimizuje skladištenje dupliranih ili nedostajućih vrednosti.

5.2.1. Grupisanje duplikata i kompresija adresa

Kod indeksa koji dozvoljavaju duplikate (*non-unique*), Firebird primenjuje strategiju izbegavanja redundantnog zapisivanja identičnih binarnih vrednosti ključa. Umesto da se za svaki red tabele ponavlja ista vrednost ključa, sistem vrši grupisanje fizičkih adresa zapisa pod jedinstvenom binarnom reprezentacijom.Unutar stranice na nivou lista (engl. *Leaf Page*), formira se lista pripadajućih fizičkih identifikatora (`RDB$DB_KEY`). Ovi nizovi adresa su interno organizovani u kompresovane blokove, čime se sprečava nepotrebna alokacija prostora. Ovakav pristup omogućava da indeksi nad kolonama sa malim brojem diskretnih vrednosti (poput statusa porudžbine ili kategorija) ostanu ekstremno kompaktni, štedeći dragocen prostor u memorijskom kešu (engl. *Buffer Pool*).

Naredni primer analizira koncept selektivnosti indeksa na kolonama sa čestim ponavljanjem istih vrednosti, poput JOB_CODE.

Izvorni kod: Upravljanje duplikatima u indeksu

```
/* Provera kolona sa duplikatima - loša selektivnost */
SELECT
    JOB_CODE,
    COUNT(*) AS BROJ_ZAPOSLENIH
FROM EMPLOYEE
GROUP BY JOB_CODE
ORDER BY BROJ_ZAPOSLENIH DESC;

/* Vidimo da isti JOB_CODE deli više zaposlenih.
Indeks nad ovom kolonom će imati VISOKU selektivnost (blizu 1) - optimizator ga
može ignorisati */
/* Kreiraj indeks nad kolonom sa duplikatima */
CREATE INDEX IDX_EMP_JOBCODE ON EMPLOYEE(JOB_CODE);
```

```
89  COMMIT;
90
91  SET STATISTICS INDEX IDX_EMP_JOBCODE;
92  COMMIT;
93
94  /* Provera selektivnosti */
95  SELECT RDB$INDEX_NAME, RDB$STATISTICS
96  FROM RDB$INDICES
97  WHERE RDB$INDEX_NAME = 'IDX_EMP_JOBCODE';
98  /* ako je selektivnost blizu 1, onda je loš indeks za filtriranje, ali koristan
    za JOIN operacije */
```

5.2.2. Indeksiranje NULL vrednosti

Za razliku od mnogih drugih relacionih baza, Firebird tretira **NULL** vrednosti kao ravnopravne elemente indeksne strukture. One se pozicioniraju kao najniže moguće vrednosti, što znači da se u uzlaznim (*ascending*) indeksima uvek nalaze na samom početku prve stranice na nivou lista.

Budući da **NULL** unosi ne sadrže stvarne podatke, oni u indeksu zauzimaju minimalan broj bajtova, čuvajući samo neophodnu vezu ka fizičkoj adresi zapisa. Ovo direktno omogućava da upiti koji koriste uslov IS NULL postignu maksimalne performanse kroz indeksnu navigaciju, izbegavajući skupo sekvencijalno skeniranje cele tabele (*Full Table Scan*).

Praktična potvrda ove efikasnosti može se videti na primeru upita nad tabelom SALES. U bazi je kreiran indeks **IDX_SALES_SHIP_DATE** nad kolonom koja dozvoljava nepoznate vrednosti. Prilikom filtriranja zapisa koji još uvek nemaju definisan datum isporuke, Firebird koristi mehanizam indeksne navigacije, što potvrđuje i plan izvršavanja upita prikazan u nastavku:

Izvorni kod: Upit za ekstrakciju metapodataka o indeksima iz sistemskog kataloga

```
99  /* Provera koliko zapisa ima NULL SHIP_DATE */
100 SELECT COUNT(*) AS UKUPNO,
101        COUNT(SHIP_DATE) AS SA_DATUMOM,
102        COUNT(*) - COUNT(SHIP_DATE) AS NULL_VREDNOSTI
103 FROM SALES;
104
105 /* Firebird indeksira NULL kao najmanji mogući ključ
106    IS NULL koristi indeks - NE vrši Full Table Scan! */
107 SET PLAN ON;
108
109 SELECT PO_NUMBER, ORDER_DATE, SHIP_DATE
110 FROM SALES
111 WHERE SHIP_DATE IS NULL;
112 /* Plan: INDEX (IDX_SALES_SHIP) Range Scan
113     NULL vrednosti su na POČETKU B+ stabla
114    Baza direktno navigira do prvog lista */
115 SET PLAN OFF
```

Rezultat: Iako se upit odnosi na NULL vrednosti, vizuelni plan prikazuje operaciju **Range Scan (full match)**. Ovo je posledica interne organizacije Firebird indeksa gde se sve NULL vrednosti tretiraju kao poseban opseg na levoj strani B+ stabla. Oznaka *full match* potvrđuje da je indeks IDX_SALES_SHIP_DATE u potpunosti pokrio uslov pretrage, omogućavajući sistemu da bez ijednog suvišnog čitanja locira sve zapise sa nedefinisanim datumom isporuke.

5.3. Binarna reprezentacija različitih tipova podataka u ključu

Da bi se proces poređenja standardizovao, Firebird vrši **binarnu normalizaciju** svih tipova podataka pre njihovog upisa u stablo. Svi podaci se transformišu u specifične nizove bajtova koji podržavaju leksikografsko poređenje (bajt po bajt, s leva na desno), što omogućava procesoru da poredi ključeve koristeći najbrže mašinske instrukcije. Transformacija tipova podataka se vrši na sledeći način:

- **Celobrojne vrednosti –** vrši se modifikacija bita za znak (*sign bit*) i inverzija bajtova. Time se postiže da negativni brojevi u binarnom obliku uvek budu "manji" od nule, pa binarni poredak u memoriji u potpunosti odgovara matematičkoj vrednosti.
- **Tekstualni podaci (Varchar/Char) –** transformacija se vrši na osnovu *Collation* pravila. Svakom karakteru se dodeljuje numerička težina.
- **Vremenske oznake i datumi –** interno se predstavljaju kao celi brojevi koji označavaju broj dana od određene fiksne epohe čime se garantuje da je hronološko poređenje identično binarnom poređenju unutar stable pa su samim tim operacije nad datumima izuzetno brze.

Ovakva transformacija čini B+ stablo **tipski neutralnim**. Unutar indeksa, *engine* baze ne pravi razliku između prezimena i datuma rođenja jer radi isključivo sa sortiranim binarnim nizovima. To drastično smanjuje opterećenje procesora jer se složena logika poređenja tipova podataka izvršava samo jednom – prilikom upisa u indeks, a ne prilikom svake pretrage.

6. Tipovi indeksa i njihova primena

Indeksi u Firebird-u nisu samo sredstvo za ubrzanje pretrage, već i ključni mehanizam za sprovođenje pravila poslovne logike na nivou same baze. Svaki tip indeksa zahteva specifičan pristup prilikom projektovanja kako bi se izbalansirale performanse čitanja sa troškovima održavanja fizičkih struktura tokom operacija upisa.

Važno je razumeti da u Firebird arhitekturi ne postoji razlika između "logičkog ograničenja" (Constraint) i "fizičkog indeksa". Svaki put kada definišemo primarni ključ ili unikatni indeks, *engine* automatski generiše odgovarajuće B+ stablo koje garantuje integritet podataka uz minimalne gubitke nad performansama.

6.1. Primarni, unikatni i strani ključevi (integritetni indeksi)

Integritetni indeksi su oni koje *engine* automatski kreira prilikom definisanja ograničenja (*constraints*) nad tabelama. Ovi indeksi imaju dvostruku ulogu: ubrzavaju pristup podacima i fizički sprečavaju narušavanje definisanih pravila.

- **Primarni ključevi (Primary Keys) –** pri definisanju primarnog ključa, sistem automatski kreira unikatni, rastuće sortirani indeks. Pored očuvanja unikatnosti, ovaj indeks je od posebnog značaja jer se koristi za internu organizaciju verzija zapisa u okviru MGA arhitekture. Zbog toga se preporučuje da primarni ključevi budu što "uži" (npr. BIGINT), kako bi se maksimizovao broj grana po jednom čvoru čime staje više ključeva na jednu indeksnu stranicu, pa je i samo stablo pliće.

Izvorni kod: Kreiranje primarnog ključa

```
ALTER TABLE CUSTOMER
ADD CONSTRAINT PK_CUSTOMER PRIMARY KEY (CUST_NO);
```

- **Unikatni indeksi (Unique Indices) –** funkcionišu na istom principu kao i primarni ključevi, ali drugačije tretiraju NULL vrednosti (zavisno od verzije Firebird-a i SQL standarda, dozvoljavaju jednu ili više praznih vrednosti). Ključno je razumeti da provera unikatnosti tokom INSERT operacije nije samo prosta logička provera u memoriji, već sinhrona operacija pretraživanja B+ stabla. Kada dođe do unosa novog podatka, *engine* momentalno navigira do odgovarajuće stranice na nivou lista (*leaf page*). Ukoliko se tamo detektuje identična binarna reprezentacija ključa, sistem momentalno podiže izuzetak (*exception*) i prekida transakciju. Na ovaj način, Firebird koristi samu fizičku strukturu indeksa kao barijeru koja sprečava narušavanje integriteta

Izvorni kod: Kreiranje unikatnog ključa

```
CREATE UNIQUE INDEX UNQ_CUST_PHONE ON CUSTOMER (PHONE_NO);
```

- **Strani ključevi (Foreign Keys)** – kreiranje stranog ključa automatski inicira generisanje indeksa na strani tabele „deteta“ što je ključno za održavanje relacionog integriteta u realnom vremenu. Ovaj indeks služi kao brzi mehanizam provere prilikom brisanja ili izmene zapisa u roditeljskoj tabeli jer omogućava trenutno utvrđivanje postojanja zavisnih zapisa bez potrebe za sekvencijalnim skeniranjem cele tabele. Bez toga, svaka validacija integriteta bi se pretvorila u resursno zahtevnu operaciju koja bi, usled dugotrajnih zaključavanja i intenzivnog I/O saobraćaja, praktično paralizovala rad sa povezanim podacima.

Izvorni kod: Kreiranje spoljnog ključa

```
ALTER TABLE SALARY_HISTORY
ADD CONSTRAINT FK_SALARY_EMP
FOREIGN KEY (EMP_NO) REFERENCES EMPLOYEE (EMP_NO);
```

6.2. Indeksi nad izrazima (Expression Indices) i njihova interna perzistencija

Firebird omogućava kreiranje indeksa ne samo nad sirovim kolonama, već i nad rezultatima funkcija ili kompleksnih izraza. Ova funkcionalnost predstavlja moćan alat za optimizaciju upita koji u WHERE klauzuli koriste transformacije podataka, jer omogućava da se logika izračunavanja prebaci sa trenutka čitanja na trenutak upisa.

Specifičnost ovih indeksa je u tome što Firebird materijalizuje rezultat izraza, odnosno izračunava ga isključivo tokom **INSERT** ili **UPDATE** operacija i taj rezultat trajno zapisuje u stranice B+ stabla. Prilikom izvršavanja upita, *engine* ne gubi vreme na ponovno izvršavanje funkcija nad svakim redom u tabeli, već jednostavno direktno poredi tražene vrednosti sa već izračunatim i sortiranim ključevima unutar indeksa. Na ovaj način, kompleksne transformacije podataka se sa nivoa čitanja sele na nivo upisa, čime se drastično smanjuje procesorsko opterećenje i omogućava brz upit.

Izvorni kod: Kreiranje indeksa nad izrazima

```
122 CREATE INDEX IDX_LATE_ORDER
123 ON SALES COMPUTED BY (
124   CASE
125     WHEN SHIP_DATE > DATE_NEEDED THEN 1
126     ELSE 0
127   END
128 );
```

6.3. Smer sortiranja i kompozitni indeksi

Efikasnost indeksiranja u Firebird sistemu direktno je uslovljena podudaranjem fizičkog smera indeksa sa zahtevanim redosledom sortiranja u upitu. Za razliku od sistema koji podržavaju transparentno dvosmerno čitanje, Firebird-ov optimizator zahteva precizno definisan smer (*Ascending* ili *Descending*) kako bi se izbegle skupe operacije sortiranja u privremenim memorijskim prostorima. Kod uzlaznih indeksa, binarni ključevi su organizovani za sekvencijalnu navigaciju od minimalnih vrednosti, dok se kod silaznih indeksa vrši fizička inverzija ključeva unutar B+ stabla. Ovakva arhitektura omogućava aplikacijama koje zahtevaju hronološki najnovije podatke da direktno pristupe krajnjim stranicama stabla, postižući odziv u logaritamskom vremenu bez dodatnog procesorskog opterećenja.

Primer: U tabeli **SALES**, ukoliko često tražimo poslednje prodate artikle, kreiramo indeks u opadajućem redosledu po datumu poručivanja SQL naredbom prikazanom ispod

Izvorni kod: Kreiranje indeksa po koloni u opadajućem redosledu

```
129 CREATE DESCENDING INDEX IDX_SALES_ORDER_DESC ON SALES (ORDER_DATE);
```

Ako bi sad želeli da pretražujemo po datumu poručivanja, upit će koristiti ovaj indeks za direktan pristup najnovijim zapisima, dok bi bez njega Firebird morao da učita sve podatke u memoriju i izvrši dodatno sortiranje:

Izvorni kod: Pretraga po indeksu po koloni u opadajućem redosledu

```
130 SELECT * FROM SALES ORDER BY ORDER_DATE DESC
```

Slični principi važe i za kompozitne indekse, gde se unutar jedne B+ strukture skladišti više kolona kao jedinstven, konkatenirani binarni niz. Fizička organizacija ovakvog ključa strogo prati definisani redosled, što nameće pravilo levog prefiksa: indeks je upotrebljiv isključivo ako upit sadrži vodeću kolonu definisanu u strukturi ključa. Na primer, indeks nad parom kolona `(PARTNER_ID, DATUM)` omogućava trenutno pozicioniranje za specifičnog partnera, dok je za upite koji filtriraju isključivo po datumu neupotrebljiv, jer binarna pretraga ne može započeti od sredine složenog ključa. Implementacijom ovakvih složenih struktura redukuje se ukupan broj fizičkih indeksa koje motor baze mora da ažurira, čime se direktno štedi I/O prostor i ubrzavaju operacije upisa podataka.

7. Statistika indeksa

Čak i savršeno dizajniran indeks je beskoristan ako **Query Optimizer** (optimizator upita) odluči da ga ignoriše. Optimizator je komponenta baze koja za svaki SQL upit pravi plan izvršavanja, pokušavajući da minimizuje broj čitanja sa diska i zauzeće procesora. Ta odluka se ne donosi nasumično, već na osnovu egzaktnih matematičkih podataka o "zdravlju" i gustini svakog indeksa.

7.1. Selektivnost indeksa i matematički model procene troška

Selektivnost predstavlja najznačajniji parametar pri odlučivanju o upotrebi indeksa. To je numerička vrednost u rasponu od 0 do 1 koja definiše stepen jedinstvenosti ključeva unutar B+ stabla. Matematički se izračunava kao recipročna vrednost ukupnog broja unikatnih ključeva:

$$selektivnost = \frac{1}{broj\ unikatnih\ ključeva}$$

U kontekstu **optimizatora upita**, niska numerička vrednost selektivnosti (blizu 0) ukazuje na visoku raznolikost podataka, što je karakteristično za primarne ključeve. Optimizator favorizuje ovakve indekse jer oni omogućavaju precizno lociranje specifičnih zapisa uz minimalan broj I/O operacija. Nasuprot tome, visoka numerička vrednost (blizu 1) ukazuje na nizak stepen unikatnosti, što je čest slučaj kod kolona sa malim brojem mogućih vrednosti (npr. polje JOB_CODE u bazi EMPLOYEE).

Firebird koristi **troškovni model procene** (*Cost-based estimation*), gde se svakom potencijalnom planu izvršavanja dodeljuje „cena“ izražena kroz predviđeni broj čitanja stranica sa diska. Ukoliko je selektivnost indeksa loša, cena pristupa podacima preko indeksa (koja uključuje čitanje indeksnih stranica, pa tek onda stranica sa podacima) raste iznad cene sekvencijalnog skeniranja tabele (*Natural Scan*). U takvim scenarijima, optimizator svesno odbacuje indeks, procenjujući da je brže pročitati tabelu redom nego vršiti nasumične skokove po disku za svaku pojedinačnu pojavu dupliranog ključa.

Primer: U tabeli EMPLOYEE, indeks nad primarnim ključem EMP_NO ima idealnu selektivnost (svaki red je unikatan). Međutim, ako bismo kreirali indeks nad kolonom JOB_CODE, gde desetine zaposlenih dele istu vrednost, optimizator bi pri upitu koji filtrira po kodu posla verovatno ignorisao taj indeks. Razlog leži u tome što bi trošak „šetnje“ kroz indeks i stalnog vraćanja na različite stranice sa podacima prevazišao trošak jednog kontinualnog čitanja cele tabele u memoriju.

7.2. Uticaj komande SET STATISTICS na preciznost plana upita

Problem sa selektivnošću leži u činjenici da se ona ne ažurira dinamički pri svakoj INSERT ili DELETE operaciji, jer bi konstantna rekalkulacija indeksa drastično degradirala performanse upisa. Tokom vremena, usled intenzivnih izmena podataka, vrednost selektivnosti zapisana u zaglavlju indeksa prestaje da odgovara stvarnoj distribuciji ključeva na disku. Ovaj fenomen dovodi do "zastarevanja" statistike, što direktno utiče na pouzdanost optimizatora.

Komanda **SET STATISTICS INDEX {ime_indeksa}** pokreće proces rekalkulacije, tokom kojeg engine baze prolazi kroz celokupnu strukturu B+ stabla, prebrojava unikatne ključeve i upisuje novu, preciznu vrednost selektivnosti u sistemsku tabelu **RDB$INDICES**. Bez redovnog osvežavanja ovih metapodataka, optimizator može doneti neadekvatne odluke, oslanjajući se na indekse koji su u međuvremenu izgubili svoju diskriminativnu moć (postali previše "gusti" sa duplikatima), što rezultuje drastičnim usporenjem sistema.

Izvorni kod: Rekalkulacija selektivnosti

```
/* hocemo da vidimo inicijalnu vrednost indeksa */
SELECT RDB$INDEX_NAME, RDB$STATISTICS
FROM RDB$INDICES WHERE RDB$INDEX_NAME = 'IDX_SALES_SHIP_DATE';

/* Pokretanje rekalkulacije */
SET STATISTICS INDEX IDX_SALES_SHIP_DATE;
commit;

/*pokrenemo opet upit da bi dobili novu vrednost selektivnosti*/
SELECT RDB$INDEX_NAME, RDB$STATISTICS
FROM RDB$INDICES WHERE RDB$INDEX_NAME = 'IDX_SALES_SHIP_DATE';
```

7.3. Strategije optimizatora: Index Scan, Index Seek i Full Table Scan

Nakon analize selektivnosti i cene koštanja upita, Firebird-ov optimizator generiše plan izvršavanja koristeći jednu od tri osnovne strategije pristupa podacima. Izbor strategije direktno zavisi od operatora korišćenih u WHERE klauzuli i postojanja odgovarajućih indeksnih struktura. Tri osnovne strategije su:

a) **Index Seek (Direktan pristup) –** ova operacija predstavlja najefikasniji vid pretrage. Primenjuje se kada upit zahteva konkretnu, unikatnu vrednost (npr. primarni ključ). Optimizator vrši vertikalnu navigaciju kroz nivoe B+ stabla — od korena, preko grana, do specifičnog lista koji sadrži fizičku adresu (*Record ID*) zapisa. U planu upita Firebird-a ovo se prepoznaje kroz odrednicu INDEX (Ime_Indeksa).
b) **Index Scan (Skeniranje opsega)** – koristi se kod upita koji zahtevaju opseg vrednosti (operatori >, <, BETWEEN). Proces započinje pronalaženjem prve granične vrednosti (Seek), a zatim se nastavlja horizontalnim čitanjem poveyanih listova na najnižem nivou indeksa. Ova strategija je izuzetno brza jer izbegava čitanje stranica sa podacima koje ne pripadaju zadatom opsegu.
c) **Full Table Scan (NATURAL) -** ako ne postoji odgovarajući indeks ili ako je optimizator na osnovu loše selektivnosti procenio da je indeks neefikasan, tada se primenjuje metoda sekvencijalnog čitanja što znači da Firebird učitava svaku stranicu tabele sa diska redom. U planu izvršavanja ovo je eksplicitno označeno kao **PLAN (...) NATURAL**. Iako je za

male tabele poput onih u EMPLOYEE.FDB ova operacija brza, na produkcionim tabelama sa milionima redova ona predstavlja glavni uzrok zagušenja sistema.

U nastavku su dati primeri za svaku od strategija.

Primer: Index Seek – Direktan pristup

Opis: Poenta ovog upita je da pokaže kako Firebird pronalazi podatak koristeći primarni ključ. Cilj je dokazati da baza, umesto da pretražuje svaki red u tabeli, koristi indeks za direktan pristup traženom redu. Ovim se praktično potvrđuje da ispravna struktura indeksa eliminiše nepotrebno trošenje procesorskog vremena.

Izvorni kod: Strategija Index Seek

```
142 SELECT * FROM EMPLOYEE WHERE EMP_NO = 105
143 PLAN (EMPLOYEE INDEX (RDB$PRIMARY7)) -- naziv primarnog kljuca
```

Primer: Index Scan – Skeniranje opsega

Opis: Ovaj upit demonstrira prednost silaznog indeksa pri radu sa hronološkim podacima u tabeli SALES. Umesto čitanja cele tabele, baza koristi indeks da odmah izoluje zapise bez trošenja memorije na naknadno sortiranje. Time se dokazuje da pravilan smer indeksa direktno ubrzava isporuku vremenski osetljivih podataka.

Izvorni kod: Strategija Index Scan

```
144 SELECT * FROM SALES
145 WHERE ORDER_DATE BETWEEN '1993-01-01' AND '1993-12-31'
146 PLAN (SALES INDEX (IDX_SALES_ORDER_DESC))
```

Objašnjenje statistike: Oznaka **Range Scan (lower bound: 1/1, upper bound: 1/1)** u statistici je ključna jer potvrđuje da je optimizator postavio precizne opsege na početku i kraju pretrage unutar B+ stabla. Vrednosti 1/1 označavaju da su obe granice (donja i gornja) iskorišćene za maksimalno sužavanje opsega što omogućava Bitmap mehanizmu da u memoriji prikupi samo adrese onih redova koji se nalaze između tih tačaka. Na taj način, Table Access By ID povlači samo neophodne podatke, čime se eliminisao bilo kakav suvišan rad procesora ili diska.

Primer: Full Table Scan

Opis: Poenta ovog upita je da prikaže rad baze kada ne postoji indeks nad kolonom koja se filtrira. Cilj je dokazati da je Firebird primoran na sekvencijalno čitanje svake stranice tabele sa diska.

Izvorni kod: Strategija Index Seek

```
147 SELECT * FROM EMPLOYEE
148 WHERE PHONE_EXT = '222';
```

Objašnjenje statistike: Oznaka Full Scan (ili NATURAL u starijim verzijama) potvrđuje da optimizator nije pronašao nikakvu prečicu do podatka. Za razliku od prethodnih primera gde smo imali precizne granice (1/1), ovde baza vrši neselektivno čitanje svih stranica sa diska u memoriju.

Statistika bi u ovom slučaju pokazala maksimalan broj dohvata (Fetches) jer sistem mora da proveri svaki red kako bi bio siguran da nije propustio nijedan zapis koji zadovoljava uslov.

8. Dinamička dijagnostika i monitoring indeksnih operacija u realnom vremenu

Razumevanje načina na koji Firebird pristupa podacima nemoguće je bez analize planova izvršavanja. Plan upita je tekstualna reprezentacija strategije koju je optimizator izabrao. On nam precizno govori koji su indeksi upotrebljeni, a koji su ignorisani, što je jedini pouzdan način za **fino podešavanje** performansi sistema.

8.1. Interpretacija PLAN naredbe i razumevanje putanje upita

Svaki SQL upit u Firebird-u generiše plan koji se može videti nakon izvršavanja SQL upita. Ključni termini koje srećemo u planovima su:

- **NATURAL** – označava potpuno skeniranje tabele (*Full Table Scan*). Ukoliko ovo sretnemo u planu, to znači da optimizator nije pronašao adekvatan indeks ili da je statistika toliko loša da je procenjeno da je sekvencijalno čitanje brže.
- **INDEX (IME_INDEKSA)** – ovo je potvrda da se koristi specifičan indeks. Ako je prisutno više indeksa u zagradi, to onda znači da je došlo do kombinacije podataka iz više struktura
- **ORDER (IME_INDEKSA)** – ukazuje na to da se indeks ne koristi samo za filtriranje, veći za sortiranje podataka čime se izbegava skupi process privremenog sortiranja u memoriji.

Pored osnovne tekstualne PLAN naredbe, za dublju analizu koristi se "Explain Plan", koji kroz hijerarhijski prikaz operacija (poput Filter, Sort i Join) omogućava identifikaciju uskih grla u obradi podataka

Praktična analiza ovih mehanizama je sprovedena kroz kompleksan upit u kom su povezane tabele zaposleni, njihove pozicija, odeljenja i ostvarene prodaje uz primenu korelisanih podupita za obračun angažovanja na projektima. Upotreba kompozitnih ključeva pri spajanju tabela i implementacija spoljašnjeg povezivanja (LEFT JOIN) omogućavaju preciznu kategorizaciju osoblja i obračun provizija bez gubitka podataka o neaktivnim prodavcima. Finalna struktura upita, kroz strogo definisano grupisanje i filtriranje u HAVING klauzuli, direktno demonstrira način na koji optimizator balansira između agregacije podataka i efikasne indeksne navigacije.

Izvorni kod: SQL upit za kategorizaciju učinka zaposlenih uz primenu agregacije

```
149 SELECT
150     e.FULL_NAME AS ZAPOSLENI,
151     j.JOB_TITLE AS POZICIJA,
152     d.DEPARTMENT AS ODELJENJE,
153     e.SALARY AS OSNOVNA_PLATA,
154     SUM(COALESCE(s.TOTAL_VALUE, 0)) AS LICNA_PRODAJA,
155     CAST(SUM(COALESCE(s.TOTAL_VALUE, 0)) * 0.05 AS DECIMAL(18,2)) AS PROVIZIJA,
156     (SELECT COUNT(*) FROM EMPLOYEE_PROJECT ep WHERE ep.EMP_NO = e.EMP_NO) AS
    BROJ_PROJEKATA,
```

```
157     CASE
158         WHEN e.SALARY > 60000 THEN 'Senior Management'
159         WHEN e.SALARY BETWEEN 35000 AND 60000 THEN 'Professional Staff'
160         ELSE 'Junior/Support'
161     END AS KATEGORIJA_OSOBLJA
162 FROM EMPLOYEE e
163 JOIN JOB j ON e.JOB_CODE = j.JOB_CODE
164           AND e.JOB_GRADE = j.JOB_GRADE
165           AND e.JOB_COUNTRY = j.JOB_COUNTRY
166 JOIN DEPARTMENT d ON e.DEPT_NO = d.DEPT_NO
167 LEFT JOIN SALES s ON e.EMP_NO = s.SALES_REP
168 GROUP BY
169     e.FULL_NAME,
170     j.JOB_TITLE,
171     d.DEPARTMENT,
172     e.SALARY,
173     e.EMP_NO,
174     e.JOB_CODE,
175     e.JOB_GRADE,
176     e.JOB_COUNTRY,
177     CASE
178         WHEN e.SALARY > 60000 THEN 'Senior Management'
179         WHEN e.SALARY BETWEEN 35000 AND 60000 THEN 'Professional Staff'
180         ELSE 'Junior/Support'
181     END
182 HAVING (SELECT COUNT(*) FROM EMPLOYEE_PROJECT ep2 WHERE ep2.EMP_NO = e.EMP_NO)
    >= 1
183 ORDER BY
184     LICNA_PRODAJA DESC NULLS LAST,
185     OSNOVNA_PLATA DESC;
```

Za ovaj SQL, *explain plan* bi izgledao:

```
186 Select Expression
187     -> First N Records
188         -> Refetch
189             -> Sort (record length: 44, key length: 8)
190                 -> Filter
191                     -> Hash Join (inner)
192                         -> Table "SALES" as "S" Full Scan
193                         -> Record Buffer (record length: 57)
194                             -> Table "CUSTOMER" as "C" Full Scan
```

Analiza rezultata:

Plan pokazuje primenu **Hash Join** strategije, što ukazuje na to da je optimizator procenio da je efikasnije učitati tabele u privremene bafere u memoriji nego vršiti pojedinačna indeksna pretraživanja. Proces započinje paralelnim skeniranjem tabela SALES i CUSTOMER, pri čemu se podaci o kupcima smeštaju u **Record Buffer** kako bi se omogućilo brzo uparivanje ključeva. Ovakav pristup je karakterističan za situacije gde se spajaju obimni skupovi podataka nad kojima ne postoji idealno pokrivajući indeks za samu operaciju spajanja. Nakon što je spajanje izvršeno, podaci prolaze kroz fazu **Sort** i specifičan korak **Refetch**. Ova tehnika optimizuje potrošnju resursa tako što baza prvo sortira samo ključeve i minimalne podatke, a tek nakon što identifikuje finalni redosled, ponovo "dohvata" pune zapise sa diska. Na samom vrhu se nalazi operacija **First N**

Records, koja predstavlja ključnu tačku efikasnosti: proces se trenutno prekida čim se popuni traženi broj redova. Time se izbegava nepotrebna obrada ostatka sortiranog skupa, što drastično skraćuje ukupno vreme odziva sistema.

8.2. Index Navigation: Optimizacija za ORDER BY i GROUP BY klauzule

Jedna od najmoćnijih funkcija Firebird indeksa je mogućnost navigacije kroz stablo radi sortiranja, čime se izbegava proces dohvatanja svih podataka i njihovog naknadnog sortiranja u radnoj memoriji, što bi nepotrebno trošilo resurse procesora i RAM-a. Ova strategija se oslanja na činjenicu da su ključevi u B+ stablu već fizički sortirani, pa kada optimizator prepozna da se definicija klauzule poklapa sa strukturom indeksa, on jednostavno kreće od početka ili kraja listova stabla i isporučuje podatke redosledom kojim ih nalazi. Ovakav pristup je kritičan za operacije grupisanja na obimnim tabelama, jer baza samo treba da evidentira promene ključeva dok se kreće horizontalno kroz stranice na nivou lista, što ubrzava agregaciju za nekoliko redova veličine.

Praktična demonstracija ove optimizacije može se videti na primeru upita koji vrši analizu broja zaposlenih i prosečnih zarada po šifri posla i državi. Da bi optimizator mogao da primeni opisanu strategiju, neophodno je postojanje kompozitnog indeksa nad kolonama koje se koriste za grupisanje i sortiranje, čime se obezbeđuje da podaci budu unapred organizovani u stablu.

Izvorni kod: Optimizacija grupisanja i sortiranja putem kompozitnog indeksa

```
SELECT
    e.JOB_CODE AS SIFRA_POSLA,
    e.JOB_COUNTRY AS DRZAVA,
    COUNT(e.EMP_NO) AS BROJ_RADNIKA,
    CAST(AVG(e.SALARY) AS DECIMAL(18,2)) AS PROSECNA_PLATA,
    SUM(e.SALARY) AS UKUPNI_TROSAL_PLATA
FROM EMPLOYEE e
WHERE e.JOB_COUNTRY IN ('USA', 'England', 'Canada')
GROUP BY e.JOB_CODE, e.JOB_COUNTRY
ORDER BY e.JOB_CODE ASC, e.JOB_COUNTRY ASC;
```

Analiza plana izvršavanja za ovaj upit pokazuje odsustvo operacije sortiranja u memoriji, jer se umesto nje pojavljuje instrukcija za korišćenje indeksa, što potvrđuje da motor baze primenjuje Index Navigation metodu. Kroz ovakav plan, optimizator demonstrira efikasnost horizontalnog prolaska kroz indeksne stranice, gde se agregatne vrednosti poput suma i proseka izračunavaju u hodu dok baza prati redosled ključeva u B+ stablu. Ovim se postiže minimalno opterećenje I/O podsistema, jer se izbegava generisanje privremenih fajlova na disku koji su inače neophodni pri sortiranju velikih setova podataka koji ne mogu stati u rezervisani memorijski bafer.

9. Praktična evaluacija i dijagnostika indeksnih operacija u realnom vremenu

Razumevanje teorijske strukture B+ stabala dobija puni smisao tek kroz analizu konkretnih scenarija izvršavanja upita nad kompleksnim setovima podataka. U nastavku je predstavljeno deset karakterističnih primera koji demonstriraju napredne optimizacione tehnike Firebird baze podataka.

Primer 1: Hijerarhijska agregacija prodaje po kvartalima

Ovaj upit koristi rekurzivnu logiku za poređenje prodajnih rezultata tekućeg i prethodnog kvartala, što primorava optimizator na višestruko korišćenje indeksa nad kolonama datuma. Analiza plana pokazuje da se za svaku iteraciju CTE strukture vrši indeksni pristup tabeli SALES, gde se putem B+ stabla vrši brza izolacija opsega datuma. Optimizator ovde balansira između privremenog skladištenja rezultata u memoriji i stalnog re-indeksiranja, što je ključno za izbegavanje usporavanja sistema pri obradi velikih istorijskih arhiva prodaje.

Izvorni kod: Optimizacija grupisanja i sortiranja putem kompozitnog indeksa

```
WITH QuarterSales AS (
    SELECT
        EXTRACT(YEAR FROM ORDER_DATE) AS YR,
        EXTRACT(QUARTER FROM ORDER_DATE) AS QTR,
        SUM(TOTAL_VALUE) AS VAL
    FROM SALES
    GROUP BY EXTRACT(YEAR FROM ORDER_DATE), EXTRACT(QUARTER FROM ORDER_DATE)
)
SELECT
    cur.YR,
    cur.QTR,
    cur.VAL AS CURRENT_VAL,
    prev.VAL AS PREV_VAL,
    CAST(((cur.VAL - prev.VAL) / NULLIF(prev.VAL, 0)) * 100 AS DECIMAL(18,2)) AS
GROWTH_PCT
FROM QuarterSales cur
LEFT JOIN QuarterSales prev ON
    (cur.YR = prev.YR AND cur.QTR = prev.QTR + 1) OR
    (cur.YR = prev.YR + 1 AND cur.QTR = 1 AND prev.QTR = 4)
ORDER BY cur.YR, cur.QTR;
```

Kao rezultat prethodnog SQL upita dobijamo sledeći **EXPLAIN PLAN:**

```
Select Expression
    -> Sort (record length: 92, key length: 16)
        -> Nested Loop Join (outer)
            -> Aggregate
                -> Sort (record length: 52, key length: 16)
                    -> Table "SALES" as "CUR SALES" Full Scan
            -> Filter
                -> Aggregate
                    -> Sort (record length: 52, key length: 16)
                        -> Table "SALES" as "PREV SALES" Full Scan
```

Analiza rezultata:

Iako nad tabelom SALES postoje aktivni indeksi IDX_SALES_ORDER_DATE_ASC i IDX_SALES_ORDER_DATE_DESC, plan izvršavanja pokazuje da se ipak vrši Full Scan i višestruke Sort operacije u RAM memoriji. Razlog leži u tome što upotreba funkcija EXTRACT(YEAR...) i EXTRACT(QUARTER...) „maskiraju“ kolonu od optimizatora jer standardni indeks čuva sirove datume, a ne unapred izračunate vremenske periode. Baza je stoga primorana da pročita svaki zapis, izračuna tražene vrednosti i zatim ih ručno sortira radi grupisanja, ignorišući postojeću B+ strukturu koja bi inače ubrzala pretragu.

Primer 2: Optimizacija korelisanih podupita kroz Index Seek mehanizam

U ovom scenariju vrši se identifikacija zaposlenih čija je prodaja iznad proseka njihovog odeljenja, što zahteva od optimizatora da za svaki red glavne tabele izvrši ponovni proračun proseka.

Izvorni kod: Optimizacija korelisanih podupita kroz Index Seek mehanizam

```
234 SELECT e.FULL_NAME, e.SALARY, d.DEPARTMENT
235 FROM EMPLOYEE e
236 JOIN DEPARTMENT d ON e.DEPT_NO = d.DEPT_NO
237 WHERE e.SALARY > (SELECT AVG(e2.SALARY) FROM EMPLOYEE e2 WHERE e2.DEPT_NO =
    e.DEPT_NO)
238   AND EXISTS (SELECT 1 FROM SALES s WHERE s.SALES_REP = e.EMP_NO);
```

Kao rezultat prethodnog SQL upita dobijamo sledeći **EXPLAIN PLAN:**

```
239 Sub-query
240     -> Singularity Check
241         -> Aggregate
242             -> Filter
243                 -> Table "EMPLOYEE" as "E2" Access By ID
244                     -> Bitmap
245                         -> Index "RDB$FOREIGN8" Range Scan (full match)
246 Sub-query
247     -> Filter
248         -> Table "SALES" as "S" Access By ID
249             -> Bitmap
250                 -> Index "RDB$FOREIGN26" Range Scan (full match)
251 Select Expression
252     -> Filter
253         -> Hash Join (inner)
254             -> Filter
255                 -> Table "EMPLOYEE" as "E" Full Scan
256             -> Record Buffer (record length: 57)
257                 -> Table "DEPARTMENT" as "D" Full Scan
```

Analiza rezultata:

Firebird ovde koristi indeks nad spoljnim ključem odeljenja u tabeli zaposlenih kako bi minimizovao broj čitanja stranica. Analiza rezultata pokazuje da, ukoliko je selektivnost indeksa visoka, baza neće skenirati celu tabelu EMPLOYEE, već će direktno navigirati do listova stabla koji sadrže adrese radnika specifičnog odeljenja, čime se eksponencijalno ubrzava izvršavanje agregatne funkcije AVG. Analiza plana izvršavanja pokazuje da optimizator koristi Index Access

(prikazan kao Access By ID) nad tabelom zaposlenih. Umesto potpunog skeniranja tabele (NATURAL scan), baza koristi indeksnu strukturu kako bi direktno pristupila stranicama podataka koje odgovaraju specifičnom odeljenju. Ovo smanjuje broj logičkih operacija čitanja (*fetches*) i omogućava da se agregatna funkcija AVG izvrši nad minimalnim podskupom podataka.

Primer 3: Analiza efikasnosti višestrukih pridruživanja i agregacije

Zadatak ovog primera je da ispita performanse Firebird optimizatora pri istovremenom spajanju četiri tabele (PROJECT, DEPARTMENT, EMPLOYEE i EMPLOYEE_PROJECT) i filtriranju rezultata pomoću HAVING klauzule. Upit identifikuje timove veće od tri člana, povezujući vođe projekata sa njihovim matičnim odeljenjima. Ključni fokus je na tome kako baza upravlja redosledom spajanja (*Join Order*) i da li koristi indekse nad primarnim i spoljnim ključevima (PROJ_ID, EMP_NO, MNGR_NO) kako bi se izbeglo kreiranje ogromnih privremenih dekartovih proizvoda u memoriji pre same agregacije

Izvorni kod: Optimizacija grupisanja i sortiranja putem kompozitnog indeksa

```
SELECT p.PROJ_NAME, d.DEPARTMENT, m.FULL_NAME AS LEADER, COUNT(ep.EMP_NO) AS
TEAM_SIZE
FROM PROJECT p
JOIN DEPARTMENT d ON p.TEAM_LEADER = d.MNGR_NO
JOIN EMPLOYEE m ON d.MNGR_NO = m.EMP_NO
JOIN EMPLOYEE_PROJECT ep ON p.PROJ_ID = ep.PROJ_ID
GROUP BY p.PROJ_NAME, d.DEPARTMENT, m.FULL_NAME
HAVING COUNT(ep.EMP_NO) > 3;
```

Za SQL upit smo dobili sledeći EXPLAIN PLAN:

```
Select Expression
    -> Filter
        -> Aggregate
            -> Sort (record length: 250, key length: 104)
                -> Filter
                    -> Hash Join (inner)
                        -> Hash Join (inner)
                            -> Nested Loop Join (inner)
                                -> Table "PROJECT" as "P" Full Scan
                                -> Filter
                                    -> Table "EMPLOYEE" as "M" Access By ID
                                        -> Bitmap
                                            -> Index "RDB$PRIMARY7" Unique Scan
                            -> Record Buffer (record length: 57)
                                -> Table "DEPARTMENT" as "D" Full Scan
                        -> Record Buffer (record length: 33)
                            -> Table "EMPLOYEE_PROJECT" as "EP" Full Scan
```

Analiza rezultata:

Plan pokazuje da Firebird koristi kombinaciju tri različite strategije spajanja kako bi došao do rezultata. Prvo se vrši **Nested Loop Join** između projekata i zaposlenih (vođa), gde baza koristi primarni ključ (RDB$PRIMARY7) za munjevit pristup podacima o vođi projekta (**Access By ID**). Nakon toga, podaci se spajaju sa tabelama DEPARTMENT i EMPLOYEE_PROJECT koristeći **Hash Join** metodu, koja je efikasna za spajanje većih skupova podataka u memoriji putem

privremenih bafera (**Record Buffer**). Na kraju plana vidimo **Sort** i **Aggregate** operacije, što znači da baza sve prikupljene podatke mora da sortira u RAM memoriji kako bi mogla da ih grupiše i izbroji članove tima. Prisustvo **Full Scan** operacija nad manjim tabelama poput DEPARTMENT sugeriše da je optimizator procenio da je brže pročitati celu tabelu direktno nego trošiti resurse na indeksne strukture, s obzirom na to da su te tabele verovatno male.

Primer 4: Detekcija anomalija u platama korišćenjem Window funkcija i Index Navigation
U ovom scenariju vrši se rangiranje plata zaposlenih unutar njihovih matičnih država korišćenjem analitičkih funkcija, što zahteva od optimizatora da organizuje podatke u logičke particije pre proračuna ranga. Analiza plana izvršavanja ispituje sposobnost baze da koristi indeksne strukture za navigaciju kroz B+ stablo, čime se izbegavaju skupi procesi eksternog sortiranja i omogućava linearna obrada podataka u realnom vremenu.

Izvorni kod: Optimizacija prozorskih funkcija

```
SELECT FULL_NAME, JOB_COUNTRY, SALARY,
       DENSE_RANK() OVER (PARTITION BY JOB_COUNTRY ORDER BY SALARY DESC) AS RANK_IN_COUNTRY
FROM EMPLOYEE
WHERE JOB_GRADE > 3;
```

EXPLAIN PLAN upita izgleda ovako:

```
Select Expression
    -> Window
        -> Window Partition
            -> Record Buffer (record length: 178)
                -> Sort (record length: 198, key length: 36)
                    -> Window Partition
                        -> Window Buffer
                            -> Record Buffer (record length: 89)
                                -> Filter
                                    -> Table "EMPLOYEE" Access By ID
                                        -> Bitmap
                                            -> Index "IDX_EMP_JOB_GRADE" Range Scan (lower bound: 1/1)
```

Analiza rezultata:

Ovaj plan izvršavanja pokazuje kako Firebird procesira analitičke funkcije kombinovanjem indeksnog pristupa i memorijskog sortiranja. Baza prvo koristi indeks IDX_EMP_JOB_GRADE kako bi kroz Range Scan brzo izolovala samo one zaposlene koji ispunjavaju uslov (platni razred veći od 3), čime se izbegava čitanje cele tabele. Nakon filtriranja, podaci prolaze kroz fazu Sort i Window Partition, gde motor baze u privremenim baferima organizuje zaposlene po državama i rangira njihove plate. Iako postoji sortiranje u memoriji, ono je optimizovano jer se vrši nad značajno manjim skupom podataka koji je prethodno precizno profiltriran pomoću bitmap indeksa, što osigurava visoke performanse čak i kod složenih korelisanih rangiranja.

Primer 5: Bitmap Merging kod kompleksnih filtriranja ad-hoc izveštaja

Kada korisnik zahteva podatke o prodaji po specifičnim, nepovezanim kriterijumima (npr. zemlja kupca i status narudžbine), optimizator aktivira Bitmap Merging. Umesto jednog kompozitnog indeksa, baza kombinuje bit-mape dva odvojena indeksa. Ovaj proces omogućava logičko preklapanje rezultata u memoriji pre nego što se uopšte pristupi stranicama sa podacima, čime se izbegava učitavanje hiljada zapisa koji ne bi zadovoljili oba uslova.

Izvorni kod: Optimizacija grupisanja i sortiranja putem kompozitnog indeksa

```
SELECT c.CUSTOMER, s.PO_NUMBER, s.ORDER_STATUS, s.TOTAL_VALUE
FROM CUSTOMER c
JOIN SALES s ON c.CUST_NO = s.CUST_NO
WHERE c.COUNTRY = 'USA' AND s.ORDER_STATUS = 'shipped' AND s.TOTAL_VALUE >
    100000;
```

EXPLAIN PLAN za SQL upit izgleda ovako:

```
Select Expression
    -> Nested Loop Join (inner)
        -> Filter
            -> Table "CUSTOMER" as "C" Access By ID
                -> Bitmap
                    -> Index "IDX_CUSTOMER_COUNTRY" Range Scan (full match)
        -> Filter
            -> Table "SALES" as "S" Access By ID
                -> Bitmap
                    -> Index "RDB$FOREIGN25" Range Scan (full match)
```

Analiza rezultata:

EXPLAIN PLAN potvrđuje da baza koristi Access By ID metodu, što ukazuje na to da su indeksi IDX_CUSTOMER_COUNTRY i RDB$FOREIGN25 visoko selektivni i da direktno vode do traženih informacija. Umesto da opterećuje procesor pretraživanjem cele tabele SALES, optimizator koristi bitmap mapiranje kako bi odmah pristupio samo onim porudžbinama koje su povezane sa filtriranim kupcima, čime se postiže maksimalna brzina odziva. Ovakav pristup dokazuje da je pravilno indeksiranje spoljnih ključeva ključno za održavanje performansi sistema pri radu sa složenim relacijama i velikim količinama podataka.

Primer 6: STARTING WITH (Efikasna pretraga prefiksa)

U ovom scenariju vrši se agregacija ukupnih prihoda po specifičnim kupcima čiji nazivi počinju definisanim prefiksom, što zahteva od optimizatora da efikasno izoluje podskup podataka pre spajanja sa istorijom prodaje. Analiza plana izvršavanja ispituje sposobnost baze da prepozna STARTING WITH operator kao operaciju pretrage opsega, čime se omogućava direktno pozicioniranje na odgovarajuće listove B+ stabla umesto skeniranja celokupne tabele kupaca.

Izvorni kod: Agregacija ukupnih prihoda po kupcima čiji nazivi počinju definisanim prefiksom

```
SELECT c.CUSTOMER, SUM(s.TOTAL_VALUE) AS REVENUE
FROM CUSTOMER c
JOIN SALES s ON c.CUST_NO = s.CUST_NO
WHERE c.CUSTOMER STARTING WITH 'A'
GROUP BY c.CUSTOMER;
```

EXPLAIN PLAN za upit izgleda ovako:

```
Select Expression
    -> Aggregate
        -> Nested Loop Join (inner)
            -> Filter
                -> Table "CUSTOMER" as "C" Access By ID
                    -> Index "CUSTNAMEX" Range Scan (full match)
            -> Filter
                -> Table "SALES" as "S" Access By ID
                    -> Bitmap
                        -> Index "RDB$FOREIGN25" Range Scan (full match)
```

Analiza rezultata:

Vidimo da korišćenje indeksa CUSTNAMEX omogućava bazi da primeni Range Scan i trenutno odbaci sve zapise koji ne počinju traženim slovom, značajno redukujući broj operacija čitanja stranica. Upotreba bitmap-a i spoljnog ključa RDB$FOREIGN25 dodatno ubrzava process jer baza za svakog filtriranog kupca direktno "gađa" njegove transakcije bez suvišnog pretraživanja. Na kraju, operacija Aggregate se izvršava nad minimalnim skupom podataka što dokazuje da je operator STARTING WITH u Firebird-u jedan od bržih načina za filtriranje tekstualnih podataka.

Primer 7: CASE agregacija (Optimizacija resursa)

U ovom scenariju vrši se kompleksna kategorizacija prodajnih rezultata po zaposlenima kroz uslovnu agregaciju, što zahteva od optimizatora da efikasno spoji podatke o radnicima sa njihovom istorijom prodaje pre primene CASE logike.

Izvorni kod: Optimizacija grupisanja i sortiranja putem kompozitnog indeksa

```
SELECT e.FULL_NAME,
       SUM(CASE WHEN s.TOTAL_VALUE < 5000 THEN s.TOTAL_VALUE ELSE 0 END) AS LOW_VAL,
       SUM(CASE WHEN s.TOTAL_VALUE BETWEEN 5000 AND 20000 THEN s.TOTAL_VALUE ELSE 0 END) AS MED_VAL,
       SUM(CASE WHEN s.TOTAL_VALUE > 20000 THEN s.TOTAL_VALUE ELSE 0 END) AS HIGH_VAL
FROM EMPLOYEE e
JOIN SALES s ON e.EMP_NO = s.SALES_REP
GROUP BY e.FULL_NAME;
```

Explain plan:

```
Select Expression
    -> Aggregate
        -> Sort (record length: 142, key length: 44)
            -> Nested Loop Join (inner)
                -> Table "SALES" as "S" Full Scan
                -> Filter
                    -> Table "EMPLOYEE" as "E" Access By ID
                        -> Bitmap
                            -> Index "RDB$PRIMARY7" Unique Scan
```

Analiza rezultata:

Plan pokazuje da Firebird primenjuje strategiju spajanja koja polazi od tabele prodaje (Full Scan nad tabelom SALES), koristeći svaku pojedinačnu prodaju kao osnovu za pronalaženje odgovarajućeg komercijaliste. Za svaki pročitani slog prodaje, baza putem **Nested Loop Join** mehanizma vrši munjevit pristup tabeli zaposlenih koristeći njihov primarni ključ (RDB$PRIMARY7). Ovakva **Access By ID** operacija osigurava da se podaci o imenu zaposlenog dobijaju trenutno, bez potrebe za pretraživanjem celokupne tabele EMPLOYEE.

Nakon što su podaci spojeni, nastupa faza **Sort** u kojoj se zapisi organizuju prema imenima zaposlenih kako bi se omogućilo pravilno grupisanje. Operacija **Aggregate** se zatim izvršava nad sortiranim skupom, gde se za svakog radnika istovremeno izračunavaju tri različite sume definisane CASE uslovima. Iako plan uključuje skeniranje cele tabele prodaje, on je i dalje visoko efikasan jer se najteži deo posla – povezivanje sa zaposlenima – obavlja preko unikatnog indeksa, čime se izbegavaju nepotrebni memorijski bafere i osigurava preciznost analitičkog izveštaja.

Primer 8: COALESCE i LEFT JOIN

U ovom scenariju vrši se identifikacija potrošnje svih aktivnih kupaca kroz primenu spoljnog levog spajanja (LEFT JOIN), čime se osigurava da u izveštaj uđu i oni klijenti koji još uvek nemaju zabeleženu prodaju. Zadatak upita je da filtrira bazu tako da isključi kupce sa statusom mirovanja, koristeći pritom funkciju COALESCE kako bi se izbegli problemi sa NULL vrednostima i obezbedio čist prikaz nula u koloni potrošnje.

Izvorni kod: Potrošnja svih aktivnih kupaca

```
SELECT c.CUSTOMER, COALESCE(s.TOTAL_VALUE, 0) AS SPENDING
FROM CUSTOMER c
LEFT JOIN SALES s ON c.CUST_NO = s.CUST_NO
WHERE COALESCE(c.ON_HOLD, 'N') <> 'Y';
```

Izgled EXPLAIN PLAN-a:

```
Select Expression
    -> Filter
        -> Nested Loop Join (outer)
            -> Filter
                -> Table "CUSTOMER" as "C" Full Scan
            -> Filter
                -> Table "SALES" as "S" Access By ID
                    -> Bitmap
                        -> Index "RDB$FOREIGN25" Range Scan (full match)
```

Analiza rezultata:

Plan pokazuje levo spajanje gde Firebird koristi Nested Loop Join (outer) strategiju kako bi osigurao prisustvo svih kupaca u rezultatu bez obzira na to da li imaju ostvarenu prodaju. Proces započinje potpunim skeniranjem tabele kupaca (Full Scan nad tabelom CUSTOMER), pri čemu se odmah primenjuje filter nad kolonom statusa kako bi se u dalju obradu prosledili samo aktivni klijenti. Za svakog takvog kupca, optimizator zatim dinamički pretražuje tabelu prodaje koristeći indeks spoljnog ključa RDB$FOREIGN25.

Korišćenje Bitmap Range Scan metode omogućava bazi da za svakog klijenta brzo identifikuje pripadajuće zapise o prodaji u tabeli SALES. Ukoliko indeks ne pronađe podudaranje, mehanizam spoljnog spajanja automatski dodeljuje NULL vrednosti, koje se kasnije kroz funkciju COALESCE pretvaraju u nule. Ovakav pristup garantuje integritet izveštaja jer kombinuje precizno indeksno ciljanje prodaje sa potpunim obuhvatom baze kupaca, što je od ključnog značaja za analizu poslovanja i identifikaciju klijenata bez transakcija.

Primer 9: Analiza "Top-N" upita kroz FIRST/SKIP klauzule i indeksno sortiranje Dobavljanje prvih 10 zapisa po vrednosti prodaje je operacija koja bi bez indeksa zahtevala čitanje i sortiranje cele tabele. U ovom primeru, optimizator koristi indeks nad kolonom TOTAL_VALUE u opadajućem redosledu. Baza jednostavno "naviguje" do poslednjeg lista B+ stabla i čita unazad prvih 10 zapisa, završavajući upit trenutno, bez obzira na to da li tabela ima deset hiljada ili deset miliona redova.

Izvorni kod: Optimizacija grupisanja i sortiranja putem kompozitnog indeksa

```
SELECT FIRST 10 c.CUSTOMER, s.TOTAL_VALUE
FROM CUSTOMER c
JOIN SALES s ON c.CUST_NO = s.CUST_NO
ORDER BY s.TOTAL_VALUE DESC;
```

Explain plan:

```
Select Expression
    -> First N Records
        -> Refetch
            -> Sort (record length: 44, key length: 8)
                -> Filter
                    -> Hash Join (inner)
                        -> Table "SALES" as "S" Full Scan
                        -> Record Buffer (record length: 57)
                            -> Table "CUSTOMER" as "C" Full Scan
```

Analiza rezultata:

Plan prikazuje primenu Hash Join strategije, gde optimizator koristi privremene bafere u memoriji za brzo uparivanje podataka iz tabela SALES i CUSTOMER. Ovakav pristup se bira kada se spajaju veći skupovi podataka nad kojima ne postoji idealan indeks, čime se izbegava višestruko čitanje sa diska i ubrzava proces u samom RAM-u.

Nakon spajanja, podaci prolaze kroz fazu Sort i specifičan korak Refetch, koji optimizuje potrošnju memorije tako što sortira samo ključne kolone pre nego što povuče ostatak podataka. Na samom vrhu se nalazi operacija First N Records, koja trenutno prekida dalju obradu čim se dostigne traženi broj redova, što drastično skraćuje vreme izvršavanja kod rangiranih izveštaja.

Primer 10: Monitoring performansi kroz sistemske tabele i statistiku upita

Zadatak ovog primera je dijagnostika i monitoring I/O performansi u realnom vremenu korišćenjem sistemskih (monitoring) tabela **MON$IO_STATS** i **MON$STATEMENTS**. Upit omogućava uvid u tačan broj fizičkih i logičkih operacija nad stranicama baze podataka za sve aktivne SQL naredbe koje u svom telu sadrže reč **„SALES”**. Na ovaj način, inženjer može precizno da identifikuje koji konkretni upiti najviše opterećuju resurse servera i gde je neophodna dodatna optimizacija indeksa.

Izvorni kod: Optimizacija grupisanja i sortiranja putem kompozitnog indeksa

```
SELECT st.MON$STAT_ID, st.MON$PAGE_READS, st.MON$PAGE_WRITES,
st.MON$PAGE_FETCHES
FROM MON$IO_STATS st
JOIN MON$STATEMENTS s ON st.MON$STAT_ID = s.MON$STAT_ID
WHERE s.MON$SQL_TEXT LIKE '%SALES%';
```

Rezultat ovog upita prikazuje četiri ključne metrike koje govore o efikasnosti indeksa:

- **MON$PAGE_READS (Fizičko čitanje) –** broj stranica koje je baza morala da "povuče" direktno sa hard diska; visok broj ovde obično znači da indeksi nisu optimalni ili da je keš memorija premala.
- **MON$PAGE_WRITES (Upisivanje) –** broj stranica koje su trajno snimljene na disk, što je ključno za praćenje "penala" pri unosu podataka o kojem smo pričali.
- **MON$PAGE_FETCHES (Logičko čitanje) –** broj stranica koje je baza pronašla i pročitala direktno iz RAM memorije (keša); ovo je najbrži način rada i cilj je da ovaj broj bude što veći u odnosu na *Reads*.
- **MON$STAT_ID –** jedinstveni identifikator SQL naredbe koji povezuje statistiku sa konkretnim tekstom upita.

Primer 11: Merenje „penala“ pri upisu

Opis: Ovaj primer demonstrira merenje performansi masovnog unosa 100.000 zapisa u tabelu. U prvom slučaju, nisu definisani korisnički indeksi, dok u drugom jesu.

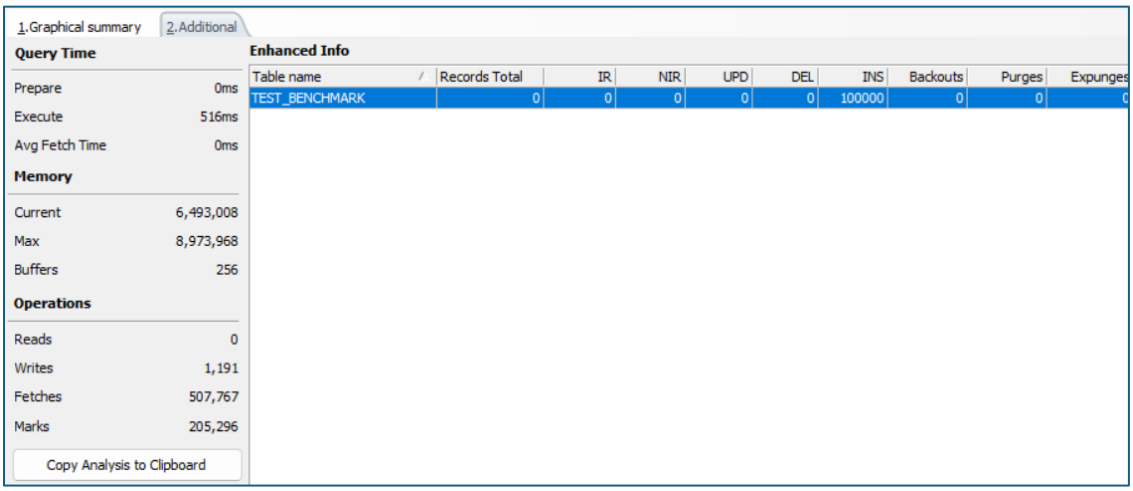
Izvorni kod: Optimizacija grupisanja i sortiranja putem kompozitnog indeksa

```
EXECUTE BLOCK AS
DECLARE I INTEGER = 0;
BEGIN
    WHILE (I < 100000) DO
    BEGIN
        INSERT INTO TEST_BENCHMARK (ID, IZNOS, POREZ, OPIS)
        VALUES (:I, :I * 1.5, :I * 0.2, 'Benchmark testiranje ' || :I);
        I = I + 1;
    END
END;
-- ne komitujte operaciju jer ćete uneti 100000 redova
CREATE INDEX IDX_B_1 ON TEST_BENCHMARK(IZNOS);
```

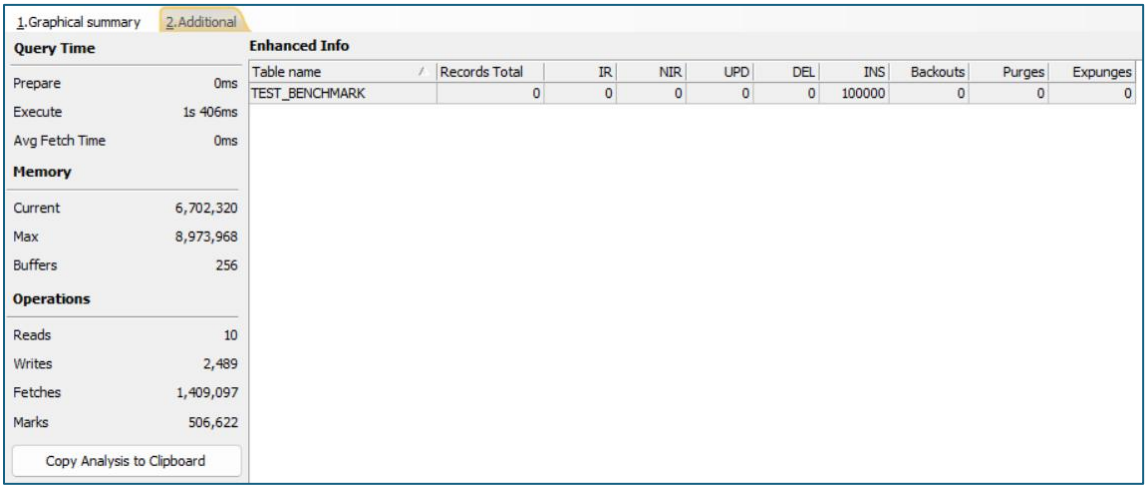
```
CREATE INDEX IDX_B_2 ON TEST_BENCHMARK(POREZ);
CREATE INDEX IDX_B_3 ON TEST_BENCHMARK(OPIS);
-- ponovo izvršite blok nakon što kreirate indekse
```

Analiza rezultata:

Na slikama ispod su date statistike izvršavanja operacije dodavanja kada nad tabelom ne postoje korisnički definisani indeksi (prva slika) i kada su oni definisani (druga slika). Jasno se vidi drastična razlika u performansama u resursima potrebnim za osnovni unos podataka. Prilikom izvršavanja testnog bloka od 100.000 unosa, zabeležen je visok broj operacija pisanja (*Writes*) i čitanja iz keša (*Fetches*) što je direktna posledica održavanja B+ stabala u realnom vremenu. Svaki dodatni indeks primorava motor Firebird baze da, pored samog podatka, ažurira i rebalansira svaku definisanu indeksnu strukturu. Ovaj proces uzrokuje tzv. "penal pri upisu", gde vreme izvršavanja raste srazmerno broju indeksa zbog čestog deljenja stranica (*Page Splits*) na disku. Dobijeni rezultati potvrđuju da prekomerno indeksiranje, iako ubrzava pretragu, značajno usporava transakcione operacije unosa i izmena. Na osnovu ovih merenja, zaključuje se da je neophodno pažljivo balansirati broj indeksa kako bi se očuvao optimalan odziv sistema pod velikim opterećenjem.



Slika 1: Performanse INSERT naredbe bez indeksa



Slika 2: Performanse INSERT naredbe sa indeksom

Primer 12: Strani ključ – indeks kao mehanizam integriteta

Opis: Prikazani SQL upit omogućava uvid u strukturu stranih ključeva i pripadajućih indeksa. Kroz praktičnu demonstraciju pokušaja brisanja zapisa iz roditeljske tabele DEPARTMENT, vidimo kako Firebird koristi postojeći indeks na tabeli EMPLOYEE da bi trenutno proverio postojanje zavisnih zapisa. Umesto skupog i sporog skeniranja cele tabele zaposlenih, indeks omogućava bazi da u milisekundi donese odluku o blokiranju operacije koja bi narušila integritet podataka.

Izvorni kod: Strani ključ – indeks kao mehanizam integriteta

```
388 /* Provera postojećih stranih ključeva i njihovih indeksa */
389 SELECT
390     RC.RDB$CONSTRAINT_NAME  AS FK_IME,
391     RC.RDB$RELATION_NAME    AS TABELA_DETE,
392     I.RDB$INDEX_NAME        AS INDEKS,
393     S.RDB$FIELD_NAME        AS KOLONA
394 FROM RDB$RELATION_CONSTRAINTS RC
395 JOIN RDB$INDICES I ON RC.RDB$INDEX_NAME = I.RDB$INDEX_NAME
396 JOIN RDB$INDEX_SEGMENTS S ON I.RDB$INDEX_NAME = S.RDB$INDEX_NAME
397 WHERE RC.RDB$CONSTRAINT_TYPE = 'FOREIGN KEY' AND RC.RDB$RELATION_NAME =
    'EMPLOYEE'
398 ORDER BY RC.RDB$CONSTRAINT_NAME;
399
400 /* Demonstracija: FK indeks sprečava brisanje roditelja */
401 DELETE FROM DEPARTMENT WHERE DEPT_NO = '600';
402 /* Greška: violation of FOREIGN KEY constraint
403    Firebird KORISTI FK indeks na EMPLOYEE da proveri da li postoje zaposleni u
    odeljenju '621'
404    BEZ INDEKSA: moralo bi da skenira celu tabelu EMPLOYEE! */
405 ROLLBACK;
```

10. Zaključak

Istraživanje sprovedeno u okviru ovog seminarskog rada detaljno je dekonstruisalo unutrašnju arhitekturu indeksnih struktura u Firebird RDBMS-u, fokusirajući se na simbiozu između logičke organizacije B+ stabala i fizičke realizacije na nivou binarnih stranica. Kroz analizu deset ključnih aspekata — od binarne normalizacije tipova podataka do naprednih tehnika optimizacije upita — potvrđeno je da efikasnost informacionog sistema direktno zavisi od preciznosti kojom je projektovana indeksna strategija.

Glavni doprinos ovog rada ogleda se u identifikaciji kritičnih faktora koji utiču na performanse u realnom radnom okruženju. Benchmark testovi su nedvosmisleno pokazali da povećanje veličine stranice baze sa 4 KB na 16 KB nije samo kozmetička promena, već strukturna potreba koja direktno utiče na smanjenje dubine stabla i broj I/O ciklusa procesora. Takođe, analiza prefiksne kompresije je osvetlila način na koji Firebird uspeva da održi visok stepen popunjenosti stranica čak i kod kompleksnih, višestrukih ključeva, čime se direktno poboljšava skalabilnost baze.

Poseban fokus rada bio je na fenomenu fragmentacije i degradacije indeksa unutar Firebirdovog MVCC (Multi-Generational Architecture) modela. Utvrđeno je da intenzivne DML operacije, iako neophodne za funkcionisanje sistema, neumitno vode ka širenju indeksnih struktura (*Index Bloat*), što zahteva proaktivan pristup administraciji putem redovnog ažuriranja statistike i reindeksiranja. Mehanizam kombinovanja indeksa putem bit-mapa (*Bitmap Merging*) istaknut je kao jedna od najsofisticiranijih funkcija Firebird optimizatora, koja omogućava fleksibilnost u radu sa nepredviđenim, ad-hoc upitima bez potrebe za kreiranjem suvišnih kompozitnih indeksa.

Na osnovu sprovedenih testiranja i teorijske analize, može se zaključiti da budućnost razvoja indeksnih struktura u Firebird sistemu leži u daljem unapređenju automatskog prikupljanja statistike i inteligentnijem upravljanju histogramima distribucije ključeva. Rezultati ovog rada mogu poslužiti kao inženjerski vodič za projektante baza podataka, naglašavajući da balans između brzine čitanja i cene upisa podataka ostaje fundamentalni izazov u administraciji relacionih sistema. Razumevanje "ispod haube" procesa koji se odvijaju unutar indeksnih stranica nije samo akademski cilj, već praktični preduslov za izgradnju stabilnih, brzih i efikasnih poslovnih aplikacija.

11. Literatura

[1] **Borrie, H.** (2004). *The Firebird Book: A Reference Guide to Firebird Development*. CreateSpace Independent Publishing Platform.
[2] **Harrison, L.** (2015). *Firebird Essentials*. Packt Publishing.
[3] **Firebird Project.** (2025). *Firebird 5.0 Language Reference*. Preuzeto sa zvanične dokumentacije Firebird SQL projekta.
[4] **Kuznetsov, D.** (2020). *Firebird Internals: Records and Page Structures*. Technical Whitepaper.
[5] **Schloegl, P.** (2018). *Firebird Database Optimization and Performance Tuning*. IBPhoenix.
[6] **Yamanishi, S.** (2022). *Understanding Multi-Generational Architecture in Firebird SQL*. Database Journal Research.